

Code Export

50 bestanden · wdd-api-scholte

Type	Aantal bestanden
.js	50

Inhoudsopgave

1. **.js** lib/auth.js
2. **.js** lib/courses.js
3. **.js** server/db.js
4. **.js** server/gameManager.js
5. **.js** server/minesState.js
6. **.js** server/mongodb.js
7. **.js** server/pokerManager.js
8. **.js** server/websocket.js
9. **.js** src/pages/api/auth/callback.js
10. **.js** src/pages/api/auth/logout.js
11. **.js** src/pages/api/balance.json.js
12. **.js** src/pages/api/buy-course.js
13. **.js** src/pages/api/check-payment.js
14. **.js** src/pages/api/courses.json.js
15. **.js** src/pages/api/create-payment.js
16. **.js** src/pages/api/daily-reward.js
17. **.js** src/pages/api/leaderboard.json.js
18. **.js** src/pages/api/login.js
19. **.js** src/pages/api/me.js
20. **.js** src/pages/api/mines/cashout.json.js
21. **.js** src/pages/api/mines/reveal.json.js
22. **.js** src/pages/api/mines/start.json.js
23. **.js** src/pages/api/payment-webhook.json.js
24. **.js** src/pages/api/rules.json.js
25. **.js** src/pages/api/score.json.js
26. **.js** src/pages/api/script.js
27. **.js** src/pages/api/signup.js
28. **.js** src/pages/api/tts.js
29. **.js** src/pages/api/webhook.js
30. **.js** src/scripts/.!98333!multiplayer.js
31. **.js** src/scripts/.!98340!blackjack.js
32. **.js** src/scripts/account.js
33. **.js** src/scripts/auth-client.js
34. **.js** src/scripts/auth-score.js
35. **.js** src/scripts/blackjack.js
36. **.js** src/scripts/client.js
37. **.js** src/scripts/index.js
38. **.js** src/scripts/leaderboard.js
39. **.js** src/scripts/login.js
40. **.js** src/scripts/logout.js

- 41. **.js** src/scripts/mines.js
- 42. **.js** src/scripts/multiplayer.js
- 43. **.js** src/scripts/poker.js
- 44. **.js** src/scripts/roulette.js
- 45. **.js** src/scripts/rules.js
- 46. **.js** src/scripts/shop-legacy.js
- 47. **.js** src/scripts/shop.js
- 48. **.js** src/scripts/signup.js
- 49. **.js** src/scripts/sound.js
- 50. **.js** src/scripts/vakken.js

■ auth.js

lib/auth.js

```
import jwt from 'jsonwebtoken';
import { ObjectId } from 'mongodb';
import { getDb } from '../server/mongodb.js';

const SESSION_SECRET = process.env.SESSION_SECRET || 'dev_secret_change_me';
const SESSION_EXPIRES = process.env.SESSION_EXPIRES || '7d';

export function createSession(user) {
  return jwt.sign(
    { sub: String(user._id), username: user.username },
    SESSION_SECRET,
    { algorithm: 'HS256', expiresIn: SESSION_EXPIRES }
  );
}

export async function getSession(tokenOrCookie) {
  if (!tokenOrCookie) return null;
  const token = tokenOrCookie.startsWith('Bearer ') ? tokenOrCookie.slice(7) : tokenOrCookie;
  try {
    const payload = jwt.verify(token, SESSION_SECRET, { algorithms: ['HS256'] });
    const db = await getDb();
    return await db.collection('users').findOne(
      { _id: new ObjectId(payload.sub) },
      { projection: { password: 0 } }
    ) || null;
  } catch {
    return null;
  }
}

export function getTokenFromRequest(request) {
  try {
    const auth = request.headers.get('authorization');
    if (auth?.startsWith('Bearer ')) return auth.slice(7);
    const cookieHeader = request.headers.get('cookie') || '';
    const cookies = Object.fromEntries(
      cookieHeader.split(';').map(s => s.trim()).filter(Boolean).map(s => {
        const eq = s.indexOf('=');
        return [s.slice(0, eq), s.slice(eq + 1)];
      })
    );
    return cookies.session || null;
  } catch {
    return null;
  }
}

export function verifyTokenRaw(token) {
  try {
    return jwt.verify(token, SESSION_SECRET, { algorithms: ['HS256'] });
  } catch {
    return null;
  }
}
```

courses.js

lib/courses.js

```
export const COURSES = [
  // Jaar 1 – Vakken
  { id: 'j1_content',          name: 'Content',          jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_emerging_tech',    name: 'Emerging Technologies', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_hci',              name: 'Human Computer Interaction', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_infoarch',        name: 'Informatiearchitectuur', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_prog',            name: 'Inleiding Programmeren', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_internetstd',      name: 'Internetstandaarden', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_maatschappij',     name: 'Maatschappij en interactie', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_npd',              name: 'New Product Development', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_ontwerpgesch',     name: 'Ontwerpgeschiedenis', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_slb',              name: 'SLB',              jaar: 1, type: 'vak',    ec: 4 },
  { id: 'j1_ucd',              name: 'User Centred Design', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_vid',              name: 'Visual Interface Design', jaar: 1, type: 'vak',    ec: 3 },
  { id: 'j1_vormgeving',       name: 'Vormgeving',       jaar: 1, type: 'vak',    ec: 3 },
  // Jaar 1 – Projecten
  { id: 'j1_proj_ind1',        name: 'Project individueel 1', jaar: 1, type: 'project', ec: 5 },
  { id: 'j1_proj_ind2',        name: 'Project individueel 2', jaar: 1, type: 'project', ec: 5 },
  { id: 'j1_proj_team1',       name: 'Project Team 1',     jaar: 1, type: 'project', ec: 5 },
  { id: 'j1_proj_team2',       name: 'Project Team 2',     jaar: 1, type: 'project', ec: 5 },
  // Jaar 2 – Vakken
  { id: 'j2_frontend',        name: 'Frontend Development', jaar: 2, type: 'vak',    ec: 4 },
  { id: 'j2_vormgeving2',      name: 'Vormgeving 2',       jaar: 2, type: 'vak',    ec: 4 },
  { id: 'j2_multimodal',       name: 'Multimodal Interaction Design', jaar: 2, type: 'vak',    ec: 3 },
  { id: 'j2_content_delivery', name: 'Content Delivery',    jaar: 2, type: 'vak',    ec: 3 },
  { id: 'j2_ontwerponderzoek', name: 'Ontwerponderzoek',    jaar: 2, type: 'vak',    ec: 3 },
  // Jaar 2 – Projecten
  { id: 'j2_proj_rmdd',        name: 'Project Responsive Multi Device Design', jaar: 2, type: 'project', ec: 6 },
  { id: 'j2_proj_beyond',      name: 'Project Beyond',     jaar: 2, type: 'project', ec: 5 },
  { id: 'j2_proj_tech',        name: 'Project Tech',       jaar: 2, type: 'project', ec: 15 },
  // Jaar 2 – Stage
  { id: 'j2_korte_stage',      name: 'Korte Stage',        jaar: 2, type: 'stage',   ec: 17 },
  // Jaar 3
  { id: 'j3_vrije_minor',      name: 'Vrije Minor',        jaar: 3, type: 'minor',   ec: 30 },
  { id: 'j3_themasemester',     name: 'Themasemester',      jaar: 3, type: 'minor',   ec: 30 },
  // Jaar 4
  { id: 'j4_lange_stage',      name: 'Lange Stage',        jaar: 4, type: 'stage',   ec: 30 },
  { id: 'j4_afstudeer',        name: 'Afstudeerproject',    jaar: 4, type: 'project', ec: 30 },
];

export const TOTAL_EC = COURSES.reduce((sum, c) => sum + c.ec, 0);
```

■ **db.js**

server/db.js

[illegible]

```
// ===== Scores =====

export async function saveScore(userId, score) {
    const db = await getDB();
    await db.collection('users').updateOne(
        { _id: new ObjectId(userId) },
        { $set: { score, scoreUpdatedAt: new Date() } }
    );
}

export async function getScore(userId) {
    const db = await getDB();
    const user = await db.collection('users').findOne(
        { _id: new ObjectId(userId) },
        { projection: { score: 1, scoreUpdatedAt: 1 } }
    );
    return user ? { score: user.score ?? 0, updated_at: user.scoreUpdatedAt ?? null } : null;
}

// ===== Courses =====

export async function getPurchasedCourses(userId) {
    const db = await getDB();
    const user = await db.collection('users').findOne(
        { _id: new ObjectId(userId) },
        { projection: { purchased_courses: 1 } }
    );
    return user?.purchased_courses ?? [];
}

export async function buyCourse(userId, courseId, price) {
    const db = await getDB();
    const result = await db.collection('users').updateOne(
        {
            _id: new ObjectId(userId),
            coins_amount: { $gte: price },
            purchased_courses: { $ne: courseId },
        },
        {
            $inc: { coins_amount: -price },
            $push: { purchased_courses: courseId },
        }
    );
    return result.modifiedCount === 1;
}

// ===== Leaderboard =====

export async function getTopPlayers(limit = 10) {
    const db = await getDB();
    return db.collection('users')
        .find({}, { projection: { username: 1, coins_amount: 1 } })
        .sort({ coins_amount: -1 })
        .limit(limit)
        .toArray();
}
```

```
export async function saveScore(userId, score) {
  const db = await getDB();
  await db.collection('users').updateOne(
    { _id: new ObjectId(userId) },
    { $set: { score, scoreUpdatedAt: new Date() } }
  );
}

export async function getScore(userId) {
  const db = await getDB();
  const user = await db.collection('users').findOne(
    { _id: new ObjectId(userId) },
    { projection: { score: 1, scoreUpdatedAt: 1 } }
  );
  return user ? { score: user.score ?? 0, updated_at: user.scoreUpdatedAt ?? null } : null;
}
```

[illegible]

```
export async function buyCourse(userId, courseId, price) {
  const db = await getDB();
  const result = await db.collection('users').updateOne(
    {
      _id: new ObjectId(userId),
      coins_amount: { $gte: price },
      purchased_courses: { $ne: courseId },
    },
    {
      $inc: { coins_amount: -price },
      $push: { purchased_courses: courseId },
    }
  );
  return result.modifiedCount === 1;
}
```

[illegible]

server/gameManager.js

```
const rooms = new Map();

const MAX_PLAYERS = 4;
const SUITS = ['♠', '♥', '♦', '♣'];
const RANKS = ['A', '2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K'];
```

```
// ■■■ Deck helpers ■■■
```

```
function createDeck() {
  const deck = [];
  for (const suit of SUITS) {
    for (const rank of RANKS) {
      deck.push({ suit, rank });
    }
  }
  return deck;
}
```

```
function shuffle(deck) {
  const shuffled = [...deck];
  for (let i = shuffled.length - 1; i > 0; i--) {
    const randomIndex = Math.floor(Math.random() * (i + 1));
    [shuffled[i], shuffled[randomIndex]] = [shuffled[randomIndex], shuffled[i]];
  }
  return shuffled;
}
```

```
function handValue(hand) {
  let total = 0;
  let aceCount = 0;

  for (const card of hand) {
    if (card.rank === 'A') {
      aceCount++;
      total += 11;
    } else if (['J', 'Q', 'K'].includes(card.rank)) {
      total += 10;
    } else {
      total += parseInt(card.rank, 10);
    }
  }
}
```

```
// Count aces as 1 instead of 11 to avoid going over 21
while (total > 21 && aceCount > 0) {
    total -= 10;
    aceCount--;
}
```

```
    return total;
}
```

[illegible]

```
function createRoom(roomId) {
  return {
    id: roomId,
    players: [],
    dealer: { hand: [], hiddenCard: null },
  };
}
```


[illegible]

```

export function snapshot(roomId) {
  const room = rooms.get(roomId);
  if (!room) return null;

  const revealDealerHand = room.phase === 'dealer' || room.phase === 'results';

  return {
    id: room.id,
    phase: room.phase,
    currentPlayerIndex: room.currentPlayerIndex,
    players: room.players.map(player => ({
      username: player.username,
      hand: player.hand,
      bet: player.bet,
      status: player.status,
      isHost: player.isHost,
      result: player.result,
      winAmount: player.winAmount,
      handValue: handValue(player.hand)
    })),
    dealer: {
      hand: room.dealer.hand,
      hasHiddenCard: !revealDealerHand && room.dealer.hiddenCard !== null,
      handValue: handValue(room.dealer.hand)
    }
  };
}

```


server/minesState.js

[illegible]

```
    return { safeCount: session.revealed.size };  
}
```

■ mongodb.js

server/mongodb.js

```
import { MongoClient } from 'mongodb';

let database = null;

export async function getDB() {
  if (database) return database;

  const uri = process.env.MONGODB_URI || import.meta.env?.MONGODB_URI;
  const dbName = process.env.DB_NAME || import.meta.env?.DB_NAME || 'blackjack';

  if (!uri) throw new Error('MONGODB_URI is not set in environment variables');

  const client = new MongoClient(uri);
  await client.connect();

  database = client.db(dbName);

  // Ensure usernames are unique
  await database.collection('users').createIndex({ username: 1 }, { unique: true });

  return database;
}

export const getDb = getDB;
```


server/pokerManager.js

```
const pokerRooms = new Map();
const MAX_PLAYERS = 6;

const SUITS = ['♠', '♥', '♦', '♣'];
const RANKS = ['2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K', 'A'];

//   Deck
```

```

    isStraight = true;
    straightTop = uniqVals[0];
}
// Wheel: A-2-3-4-5
if (vals[0] === 14 && vals[1] === 5 && vals[2] === 4 && vals[3] === 3 && vals[4] === 2) {
    isStraight = true;
    straightTop = 5;
}

const freq = {};
for (const v of vals) freq[v] = (freq[v] || 0) + 1;
const groups = Object.entries(freq)
    .map(([v, c]) => [v, c])
    .sort((a, b) => b[1] - a[1] || b[0] - a[0]);
const freqs = groups.map(g => g[1]);
const groupVals = groups.map(g => g[0]);

let rank, name, tiebreakers;

if (isFlush && isStraight) {
    rank = straightTop === 14 ? 9 : 8;
    name = straightTop === 14 ? 'Royal Flush' : 'Straight Flush';
    tiebreakers = [straightTop];
} else if (freqs[0] === 4) {
    rank = 7; name = 'Four of a Kind'; tiebreakers = [...groupVals];
} else if (freqs[0] === 3 && freqs[1] === 2) {
    rank = 6; name = 'Full House'; tiebreakers = [...groupVals];
} else if (isFlush) {
    rank = 5; name = 'Flush'; tiebreakers = [...vals];
} else if (isStraight) {
    rank = 4; name = 'Straight'; tiebreakers = [straightTop];
} else if (freqs[0] === 3) {
    rank = 3; name = 'Three of a Kind'; tiebreakers = [...groupVals];
} else if (freqs[0] === 2 && freqs[1] === 2) {
    rank = 2; name = 'Two Pair'; tiebreakers = [...groupVals];
} else if (freqs[0] === 2) {
    rank = 1; name = 'Pair'; tiebreakers = [...groupVals];
} else {
    rank = 0; name = 'High Card'; tiebreakers = [...vals];
}

return { rank, name, tiebreakers };
}

function compareHands(a, b) {
    if (a.rank !== b.rank) return a.rank - b.rank;
    for (let i = 0; i < Math.min(a.tiebreakers.length, b.tiebreakers.length); i++) {
        if (a.tiebreakers[i] !== b.tiebreakers[i]) return a.tiebreakers[i] - b.tiebreakers[i];
    }
    return 0;
}

export function getBestHand(holeCards, community) {
    const all = [...holeCards, ...community];
    if (all.length < 5) return { rank: -1, name: '-', tiebreakers: [] };
    const combos = combinations(all, 5);
    let best = null;
    for (const combo of combos) {
        const h = evaluateFive(combo);
        if (!best || compareHands(h, best) > 0) best = h;
    }
    return best;
}

```



```

const current = room.players[room.currentPlayerIndex];
if (!current || current.clientId !== clientId) return { error: 'Niet jouw beurt' };

current.status = 'folded';
current.hasActed = true;

advanceBettingTurn(room);
return { ok: true };
}

export function playerCheck(roomId, clientId) {
  const room = pokerRooms.get(roomId);
  if (!room) return { error: 'Tafel niet gevonden' };
  if (!['preflop', 'flop', 'turn', 'river'].includes(room.phase)) return { error: 'Kan nu niet checken' };

  const current = room.players[room.currentPlayerIndex];
  if (!current || current.clientId !== clientId) return { error: 'Niet jouw beurt' };
  if (current.streetBet < room.currentBet) return { error: 'Je kunt niet checken – er is een bod' };

  current.hasActed = true;
  advanceBettingTurn(room);
  return { ok: true };
}

export function playerCall(roomId, clientId) {
  const room = pokerRooms.get(roomId);
  if (!room) return { error: 'Tafel niet gevonden' };
  if (!['preflop', 'flop', 'turn', 'river'].includes(room.phase)) return { error: 'Kan nu niet callen' };

  const current = room.players[room.currentPlayerIndex];
  if (!current || current.clientId !== clientId) return { error: 'Niet jouw beurt' };

  const callAmount = room.currentBet - current.streetBet;
  if (callAmount <= 0) return { error: 'Niets te callen – gebruik check' };

  current.streetBet = room.currentBet;
  current.totalBet += callAmount;
  room.pot += callAmount;
  current.hasActed = true;

  advanceBettingTurn(room);
  return { ok: true, callAmount };
}

export function playerRaise(roomId, clientId, raiseBy) {
  const room = pokerRooms.get(roomId);
  if (!room) return { error: 'Tafel niet gevonden' };
  if (!['preflop', 'flop', 'turn', 'river'].includes(room.phase)) return { error: 'Kan nu niet raisen' };

  const current = room.players[room.currentPlayerIndex];
  if (!current || current.clientId !== clientId) return { error: 'Niet jouw beurt' };
  if (raiseBy < 1) return { error: 'Raise moet minimaal 1 zijn' };

  const callAmount = room.currentBet - current.streetBet;
  const totalAdd = callAmount + raiseBy;

  current.streetBet += totalAdd;
  current.totalBet += totalAdd;
  room.pot += totalAdd;
  room.currentBet = current.streetBet;
  current.hasActed = true;
}

```



```

}

function findNextToAct(room) {
  const start = room.currentPlayerIndex;
  for (let i = 1; i <= room.players.length; i++) {
    const idx = (start + i) % room.players.length;
    const p = room.players[idx];
    if (p.status !== 'playing') continue;
    if (!p.hasActed || p.streetBet < room.currentBet) return idx;
  }
  return start;
}

function advanceStreet(room) {
  for (const player of room.players) {
    if (player.status === 'playing') {
      player.streetBet = 0;
      player.hasActed = false;
    }
  }
  room.currentBet = 0;

  if (room.phase === 'preflop') {
    room.community.push(room.deck.pop(), room.deck.pop(), room.deck.pop());
    room.phase = 'flop';
  } else if (room.phase === 'flop') {
    room.community.push(room.deck.pop());
    room.phase = 'turn';
  } else if (room.phase === 'turn') {
    room.community.push(room.deck.pop());
    room.phase = 'river';
  } else if (room.phase === 'river') {
    resolveShowdown(room);
    return;
  }

  room.currentPlayerIndex = findNextActive(room, room.dealerIndex);
}

function resolveShowdown(room) {
  room.phase = 'results';
  room.winsCredited = false;

  const activePlayers = room.players.filter(p => p.status === 'playing');

  for (const player of activePlayers) {
    player.handResult = getBestHand(player.hand, room.community);
  }

  let best = null;
  for (const player of activePlayers) {
    if (!best || compareHands(player.handResult, best) > 0) {
      best = player.handResult;
    }
  }

  const winners = activePlayers.filter(p => compareHands(p.handResult, best) === 0);
  const winShare = Math.floor(room.pot / winners.length);

  for (const player of room.players) {
    if (winners.find(w => w.clientId === player.clientId)) {
      player.result = winners.length > 1 ? 'tie' : 'win';
    }
  }
}

```


■ websocket.js

server/websocket.js

```
import { WebSocketServer } from 'ws';
import { verifyTokenRaw } from '../lib/auth.js';
import * as gameManager from './gameManager.js';
import * as pokerManager from './pokerManager.js';
import { creditCoins, deductCoins } from './db.js';

const clients = new Map();

function generateClientId() {
  return Math.random().toString(36).slice(2, 10);
}

export function attachWebSocket(httpServer) {
  const wss = new WebSocketServer({ server: httpServer });

  wss.on('connection', (socket) => {
    const clientId = generateClientId();
    clients.set(clientId, { socket, roomId: null, game: null, userId: null, username: null });

    safeSend(socket, { type: 'WELCOME', clientId });

    socket.on('message', async (raw) => {
      let message;
      try {
        message = JSON.parse(String(raw));
      } catch {
        return;
      }
      await handleMessage(clientId, message);
    });

    socket.on('close', async () => {
      const client = clients.get(clientId);

      if (client?.roomId) {
        if (client.game === 'poker') {
          pokerManager.removePlayer(client.roomId, clientId);
          broadcastPokerRoomUpdate(client.roomId);
        } else {
          gameManager.removePlayer(client.roomId, clientId);
          broadcastRoomUpdate(client.roomId);
        }
      }

      clients.delete(clientId);
    });
  });

  // Message handler
  async function handleMessage(clientId, message) {
    if (!message?.type) return;

    const client = clients.get(clientId);
    if (!client) return;

    switch (message.type) {
      // Blackjack
    }
  }
}
```



```

        sendError(clientId, 'Ongeldig kamer-nummer (alleen cijfers)');
        return;
    }

    if (client.roomId) {
        if (client.game === 'poker') {
            pokerManager.removePlayer(client.roomId, clientId);
            broadcastPokerRoomUpdate(client.roomId);
        } else {
            gameManager.removePlayer(client.roomId, clientId);
            broadcastRoomUpdate(client.roomId);
        }
    }

    client.roomId = roomId;
    client.game = 'blackjack';
    client.userId = payload.sub;
    client.username = payload.username;

    const result = gameManager.addPlayer(roomId, clientId, payload.sub, payload.username);
    if (result.error) {
        client.roomId = null;
        client.game = null;
        sendError(clientId, result.error);
        return;
    }

    broadcastRoomUpdate(roomId);
}

function handleStartGame(clientId, client) {
    if (!client.roomId) return;

    const result = gameManager.startGame(client.roomId, clientId);
    if (result.error) {
        sendError(clientId, result.error);
        return;
    }

    broadcastRoomUpdate(client.roomId);
}

async function handlePlaceBet(clientId, client, message) {
    if (!client.roomId) return;

    const amount = parseInt(message.amount, 10);
    if (!amount || amount < 1) {
        sendError(clientId, 'Ongeldige inzet');
        return;
    }

    const coinsDeducted = await deductCoins(client.userId, amount);
    if (!coinsDeducted) {
        sendError(clientId, 'Niet genoeg coins');
        return;
    }

    const result = gameManager.placeBet(client.roomId, clientId, amount);
    if (result.error) {
        await creditCoins(client.userId, amount);
        sendError(clientId, result.error);
        return;
    }
}

```

[illegible]

```

const roomId = String(message.roomId || '').trim();
if (!roomId || !/^d+$/i.test(roomId)) {
  sendError(clientId, 'Ongeldig kamer-nummer (alleen cijfers)');
  return;
}

if (client.roomId) {
  if (client.game === 'poker') {
    pokerManager.removePlayer(client.roomId, clientId);
    broadcastPokerRoomUpdate(client.roomId);
  } else {
    gameManager.removePlayer(client.roomId, clientId);
    broadcastRoomUpdate(client.roomId);
  }
}

client.roomId = roomId;
client.game = 'poker';
client.userId = payload.sub;
client.username = payload.username;

const result = pokerManager.addPlayer(roomId, clientId, payload.sub, payload.username);
if (result.error) {
  client.roomId = null;
  client.game = null;
  sendError(clientId, result.error);
  return;
}

broadcastPokerRoomUpdate(roomId);
}

function handlePokerStartGame(clientId, client) {
  if (!client.roomId) return;
  const result = pokerManager.startGame(client.roomId, clientId);
  if (result.error) { sendError(clientId, result.error); return; }
  broadcastPokerRoomUpdate(client.roomId);
}

async function handlePokerPlaceAnte(clientId, client, message) {
  if (!client.roomId) return;

  const amount = parseInt(message.amount, 10);
  if (!amount || amount < 1) { sendError(clientId, 'Ongeldige inzet'); return; }

  const ok = await deductCoins(client.userId, amount);
  if (!ok) { sendError(clientId, 'Niet genoeg coins'); return; }

  const result = pokerManager.placeAnte(client.roomId, clientId, amount);
  if (result.error) {
    await creditCoins(client.userId, amount);
    sendError(clientId, result.error);
    return;
  }

  const room = pokerManager.getRoom(client.roomId);
  if (room?.phase === 'results') await creditPokerWinnings(room);

  broadcastPokerRoomUpdate(client.roomId);
}

async function handlePokerFold(clientId, client) {

```

```

    if (!client.roomId) return;
    const result = pokerManager.playerFold(client.roomId, clientId);
    if (result.error) { sendError(clientId, result.error); return; }

    const room = pokerManager.getRoom(client.roomId);
    if (room?.phase === 'results') await creditPokerWinnings(room);

    broadcastPokerRoomUpdate(client.roomId);
}

async function handlePokerCheck(clientId, client) {
    if (!client.roomId) return;
    const result = pokerManager.playerCheck(client.roomId, clientId);
    if (result.error) { sendError(clientId, result.error); return; }

    const room = pokerManager.getRoom(client.roomId);
    if (room?.phase === 'results') await creditPokerWinnings(room);

    broadcastPokerRoomUpdate(client.roomId);
}

async function handlePokerCall(clientId, client) {
    if (!client.roomId) return;

    const room = pokerManager.getRoom(client.roomId);
    const player = room?.players.find(p => p.clientId === clientId);
    if (!room || !player) return;

    const callAmount = room.currentBet - player.streetBet;
    if (callAmount > 0) {
        const ok = await deductCoins(client.userId, callAmount);
        if (!ok) { sendError(clientId, 'Niet genoeg coins'); return; }
    }

    const result = pokerManager.playerCall(client.roomId, clientId);
    if (result.error) {
        if (callAmount > 0) await creditCoins(client.userId, callAmount);
        sendError(clientId, result.error);
        return;
    }

    const updatedRoom = pokerManager.getRoom(client.roomId);
    if (updatedRoom?.phase === 'results') await creditPokerWinnings(updatedRoom);

    broadcastPokerRoomUpdate(client.roomId);
}

async function handlePokerRaise(clientId, client, message) {
    if (!client.roomId) return;

    const raiseBy = parseInt(message.raiseBy, 10);
    if (!raiseBy || raiseBy < 1) { sendError(clientId, 'Ongeldige raise'); return; }

    const room = pokerManager.getRoom(client.roomId);
    const player = room?.players.find(p => p.clientId === clientId);
    if (!room || !player) return;

    const callAmount = room.currentBet - player.streetBet;
    const totalCost = callAmount + raiseBy;

    const ok = await deductCoins(client.userId, totalCost);
    if (!ok) { sendError(clientId, 'Niet genoeg coins'); return; }
}

```



```

const players = room.players.map(p => {
  const isMe = p.username === client.username;
  const showHand = isMe || (revealHands && p.status !== 'folded');
  const hasCards = ['playing', 'folded'].includes(p.status);

  let handResult = null;
  if (revealHands) {
    handResult = p.handResult;
  } else if (isMe && p.hand?.length >= 2 && room.community.length >= 3) {
    handResult = pokerManager.getBestHand(p.hand, room.community);
  }

  return {
    username: p.username,
    hand: showHand ? p.hand : [],
    hasCards,
    ante: p.ante,
    streetBet: p.streetBet,
    totalBet: p.totalBet,
    status: p.status,
    isHost: p.isHost,
    result: p.result,
    winAmount: p.winAmount,
    handResult,
  };
});

safeSend(client.socket, {
  type: 'POKER_ROOM_UPDATE',
  id: room.id,
  phase: room.phase,
  pot: room.pot,
  currentBet: room.currentBet,
  currentPlayerIndex: room.currentPlayerIndex,
  community: room.community,
  players,
  myUsername: client.username,
});
}
}

function sendError(clientId, message) {
  const client = clients.get(clientId);
  if (client) {
    safeSend(client.socket, { type: 'ERROR', message });
  }
}

function safeSend(socket, message) {
  try {
    socket.send(JSON.stringify(message));
  } catch {
    // Client disconnected mid-send, ignore
  }
}
}

```

■ callback.js

src/pages/api/auth/callback.js

```
export async function GET(){  
  return new Response('Auth callback not used. Use /login and /signup instead.', { status: 410 });  
}
```

■ logout.js

src/pages/api/auth/logout.js

```
export async function GET({ cookies, redirect }){
  cookies.delete('session', { path: '/' });
  return redirect('/');
}
```

■ balance.json.js

src/pages/api/balance.json.js

```
import { getSession, getTokenFromRequest } from '../../../lib/auth.js';
import { getBalance, setBalance } from '../../../server/db.js';

export async function GET({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });
    const balance = await getBalance(String(user._id));
    return new Response(JSON.stringify({ balance }), { status: 200 });
  } catch (e) {
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });
    const body = await request.json();
    const amount = Number(body.balance);
    if (!Number.isFinite(amount) || amount <= 0) {
      return new Response(JSON.stringify({ error: 'invalid balance' }), { status: 400 });
    }
    await setBalance(String(user._id), amount);
    return new Response(JSON.stringify({ ok: true }), { status: 200 });
  } catch (e) {
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}
```

■ buy-course.js

src/pages/api/buy-course.js

```
import { getSession, getTokenFromRequest } from '../../../lib/auth.js';
import { buyCourse } from '../../../server/db.js';
import { COURSES } from '../../../lib/courses.js';

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });

    const { courseId } = await request.json();
    const course = COURSES.find(c => c.id === courseId);
    if (!course) return new Response(JSON.stringify({ error: 'Onbekend vak' }), { status: 400 });

    const ok = await buyCourse(String(user._id), courseId, course.ec);
    if (!ok) {
      return new Response(JSON.stringify({ error: 'Niet genoeg EC-Punten of al gekocht' }), { status: 400 });
    }

    return new Response(JSON.stringify({ ok: true }), { status: 200 });
  } catch (e) {
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}
```

■ check-payment.js

src/pages/api/check-payment.js

```
import { createMollieClient } from '@mollie/api-client';
import { getSession, getTokenFromRequest } from '../../lib/auth.js';
import { getPaymentByMollieId, markPaymentPaid, creditCoins } from '../../server/db.js';

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) {
      return new Response(JSON.stringify({ error: 'Niet ingelogd' }), { status: 401 });
    }

    const { molliePaymentId } = await request.json();
    if (!molliePaymentId) {
      return new Response(JSON.stringify({ error: 'Geen betaling id' }), { status: 400 });
    }

    const existing = await getPaymentByMollieId(molliePaymentId);

    if (!existing) {
      return new Response(JSON.stringify({ error: 'Betaling niet gevonden' }), { status: 404 });
    }

    // Controleer of de betaling bij de ingelogde gebruiker hoort
    if (String(existing.userId) !== String(user._id)) {
      return new Response(JSON.stringify({ error: 'Niet geautoriseerd' }), { status: 403 });
    }

    // Al verwerkt
    if (existing.status === 'paid') {
      return new Response(JSON.stringify({ credited: false, alreadyPaid: true }), { status: 200 });
    }

    // Controleer de status rechtstreeks bij Mollie
    const mollie = createMollieClient({ apiKey: import.meta.env.MOLLIE_API_KEY });
    const molliePayment = await mollie.payments.get(molliePaymentId);

    if (molliePayment.status !== 'paid') {
      return new Response(JSON.stringify({ credited: false }), { status: 200 });
    }

    // Coins bijschrijven
    await markPaymentPaid(molliePaymentId);
    await creditCoins(existing.userId, existing.coins);

    console.log('check-payment: coins bijgeschreven', existing.coins, 'voor gebruiker', existing.userId);

    return new Response(JSON.stringify({ credited: true, coins: existing.coins }), { status: 200 });
  } catch (e) {
    console.error('check-payment fout:', e.message);
    return new Response(JSON.stringify({ error: e.message }), { status: 500 });
  }
}
```

■ courses.json.js

src/pages/api/courses.json.js

```
import { getSession, getTokenFromRequest } from '../../../lib/auth.js';
import { getPurchasedCourses } from '../../../server/db.js';
import { COURSES, TOTAL_EC } from '../../../lib/courses.js';

export async function GET({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });

    const purchased = await getPurchasedCourses(String(user._id));
    return new Response(JSON.stringify({ courses: COURSES, purchased, totalEc: TOTAL_EC }), { status: 200 });
  } catch (e) {
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}
```

■ create-payment.js

src/pages/api/create-payment.js

```
import { createMollieClient } from '@mollie/api-client';
import { getSession, getTokenFromRequest } from '../../lib/auth.js';
import { createPayment } from '../../server/db.js';

const PACKAGES = {
  p1: { coins: 60, price: '2.60', label: '60 EC-Punten voor €2,60' },
  p5: { coins: 120, price: '5.10', label: '120 EC-Punten voor €5,10' },
  p10: { coins: 180, price: '7.60', label: '180 EC-Punten voor €7,60' }
};

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) return new Response(JSON.stringify({ error: 'Niet ingelogd' }), { status: 401 });

    const { packageId } = await request.json();
    const pkg = PACKAGES[packageId];
    if (!pkg) return new Response(JSON.stringify({ error: 'Ongeldig pakket' }), { status: 400 });

    const mollie = createMollieClient({ apiKey: import.meta.env.MOLLIE_API_KEY });

    // Bepaal de base URL vanuit de binnenkomende request zodat dit
    // automatisch werkt op zowel localhost als productie (Render).
    // Render zet x-forwarded-proto op 'https'; lokaal valt het terug op 'http'.
    const proto = request.headers.get('x-forwarded-proto') || 'http';
    const host = request.headers.get('host');
    const baseUrl = proto + '://' + host;

    const isLocalhost = host.includes('localhost') || host.includes('127.0.0.1');

    const payment = await mollie.payments.create({
      amount: { currency: 'EUR', value: pkg.price },
      description: pkg.label,
      method: 'ideal',
      redirectUrl: baseUrl + '/shop?payment=success',
      ...(isLocalhost & { webhookUrl: baseUrl + '/api/webhook' }),
      metadata: { packageId, userId: String(user._id) }
    });

    await createPayment({
      molliePaymentId: payment.id,
      userId: String(user._id),
      packageId,
      coins: pkg.coins,
      status: 'pending',
      createdAt: new Date()
    });

    return new Response(JSON.stringify({ paymentUrl: payment.getCheckoutUrl(), molliePaymentId: payment.id }));
  } catch (e) {
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}
```

■ daily-reward.js

src/pages/api/daily-reward.js

```
import { getSession, getTokenFromRequest } from '../../../lib/auth.js';
import { getDB } from '../../../server/mongodb.js';

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });

    const now = new Date();
    const lastReward = user.lastDailyReward ? new Date(user.lastDailyReward) : null;
    const sameDay = lastReward && lastReward.toISOString().slice(0, 10) === now.toISOString().slice(0, 10);

    if (sameDay) {
      const tomorrow = new Date(now);
      tomorrow.setUTCDate(tomorrow.getUTCDate() + 1);
      tomorrow.setUTCHours(0, 0, 0, 0);
      return new Response(JSON.stringify({
        claimed: false,
        message: 'Al geclaimd vandaag',
        nextClaimAt: tomorrow.toISOString()
      }), { status: 200 });
    }

    const db = await getDB();
    await db.collection('users').updateOne(
      { _id: user._id },
      { $inc: { coins_amount: 1 }, $set: { lastDailyReward: now } }
    );

    return new Response(JSON.stringify({
      claimed: true,
      coinsAdded: 1,
      message: '+1 coins geclaimd!'
    }), { status: 200 });
  } catch (e) {
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}
```

■ leaderboard.json.js

src/pages/api/leaderboard.json.js

```
import { getTopPlayers } from '../../../server/db.js';

export async function GET() {
  try {
    const players = await getTopPlayers(10);
    const top = players.map((p, i) => ({
      rank: i + 1,
      username: p.username,
      coins_amount: p.coins_amount ?? 0,
    }));
    return new Response(JSON.stringify({ top }), { status: 200 });
  } catch (e) {
    console.error('leaderboard err', e?.message);
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}
```

■ login.js

src/pages/api/login.js

```
import bcrypt from 'bcryptjs';
import { getDB } from '../../../server/mongodb.js';
import { createSession } from '../../../lib/auth.js';

export async function POST({ request }) {
  try {
    const { username, password } = await request.json();

    if (!username || !password) {
      return new Response(
        JSON.stringify({ message: 'Gebruikersnaam en wachtwoord zijn verplicht' }),
        { status: 400 }
      );
    }

    const db = await getDB();
    const user = await db.collection('users').findOne({ username });

    if (!user) {
      return new Response(
        JSON.stringify({ message: 'Gebruiker niet gevonden' }),
        { status: 401 }
      );
    }

    const passwordIsCorrect = await bcrypt.compare(password, user.password);
    if (!passwordIsCorrect) {
      return new Response(
        JSON.stringify({ message: 'Verkeerd wachtwoord' }),
        { status: 401 }
      );
    }

    // Give 1000 bonus coins if the user hasn't logged in today yet
    const dailyRewardGiven = await givesDailyRewardIfEligible(db, user);

    const token = createSession(user);

    return new Response(
      JSON.stringify({ message: 'Ingelogd', token, dailyReward: dailyRewardGiven }),
      {
        status: 200,
        headers: { 'Set-Cookie': `session=${token}; Path=/; HttpOnly; SameSite=Lax; Max-Age=604800` }
      }
    );
  } catch (error) {
    return new Response(
      JSON.stringify({ message: 'Server fout: ' + error?.message }),
      { status: 500 }
    );
  }
}

async function givesDailyRewardIfEligible(db, user) {
  const today = new Date().toISOString().slice(0, 10); // "YYYY-MM-DD"
  const lastRewardDate = user.lastDailyReward
    ? new Date(user.lastDailyReward).toISOString().slice(0, 10)
    : null;
}
```

```
const alreadyReceivedToday = lastRewardDate === today;
if (alreadyReceivedToday) return false;

await db.collection('users').updateOne(
  { _id: user._id },
  { $inc: { coins_amount: 1 }, $set: { lastDailyReward: new Date() } } // 1 EC-punt staat gelijk aan ongeveer
);

return true;
}
```

```
import { getSession, getTokenFromRequest } from '../../../lib/auth.js';

export async function GET({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if (!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });
    return new Response(JSON.stringify({
      id: String(user._id),
      username: user.username,
      coins_amount: user.coins_amount ?? 0,
      createdAt: user.createdAt || null
    }), { status: 200 });
  } catch (e) {
    return new Response(JSON.stringify({ error: e?.message }), { status: 500 });
  }
}
```

■ cashout.json.js

src/pages/api/mines/cashout.json.js

```
import { getSession as getAuthSession, getTokenFromRequest } from '../../../lib/auth.js';
import { getSession as getMinesSession, cashOut, calcMultiplier } from '../../../server/minesState.js';
import { creditCoins } from '../../../server/db.js';

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getAuthSession(token);
    if (!user) return json({ error: 'unauthorized' }, 401);

    const { gameId } = await request.json();

    const session = getMinesSession(gameId);
    if (!session) return json({ error: 'Spel niet gevonden' }, 404);
    if (session.userId !== String(user._id)) return json({ error: 'Onbevoegd' }, 403);
    if (session.status !== 'active') return json({ error: 'Spel al afgelopen' }, 400);
    if (session.revealed.size === 0) return json({ error: 'Onthuld eerst een tegel' }, 400);

    const result = cashOut(gameId);
    if (!result) return json({ error: 'Cashout mislukt' }, 500);

    const multiplier = calcMultiplier(session.mineCount, result.safeCount);
    const winAmount = Math.floor(session.bet * multiplier);

    await creditCoins(session.userId, winAmount);

    return json({
      winAmount,
      multiplier,
      minePositions: [...session.minePositions],
    });
  } catch (e) {
    return json({ error: e?.message }, 500);
  }
}

function json(data, status = 200) {
  return new Response(JSON.stringify(data), {
    status,
    headers: { 'Content-Type': 'application/json' },
  });
}
```

■ reveal.json.js

src/pages/api/mines/reveal.json.js

```
import { getSession as getAuthSession, getTokenFromRequest } from '../../../lib/auth.js';
import { getSession as getMinesSession, revealTile, calcMultiplier } from '../../../server/minesState.js';

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getAuthSession(token);
    if (!user) return json({ error: 'unauthorized' }, 401);

    const { gameId, index } = await request.json();

    if (typeof index !== 'number' || index < 0 || index > 24) return json({ error: 'Ongeldig vakje' }, 400);

    const session = getMinesSession(gameId);
    if (!session) return json({ error: 'Spel niet gevonden' }, 404);
    if (session.userId !== String(user._id)) return json({ error: 'Onbevoegd' }, 403);
    if (session.status !== 'active') return json({ error: 'Spel al afgelopen' }, 400);
    if (session.revealed.has(index)) return json({ error: 'Al onthuld' }, 400);

    const result = revealTile(gameId, index);
    if (!result) return json({ error: 'Fout bij onthullen' }, 500);

    if (result.mine) {
      return json({ mine: true, minePositions: [...session.minePositions] });
    }

    const multiplier = calcMultiplier(session.mineCount, result.safeCount);
    return json({ safe: true, safeCount: result.safeCount, multiplier });
  } catch (e) {
    return json({ error: e?.message }, 500);
  }
}

function json(data, status = 200) {
  return new Response(JSON.stringify(data), {
    status,
    headers: { 'Content-Type': 'application/json' },
  });
}
```

■ start.json.js

src/pages/api/mines/start.json.js

```
import { getSession as getAuthSession, getTokenFromRequest } from '../../../lib/auth.js';
import { deductCoins } from '../../../server/db.js';
import { createSession, calcMultiplier } from '../../../server/minesState.js';

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getAuthSession(token);
    if (!user) return json({ error: 'unauthorized' }, 401);

    const body = await request.json();
    const bet = parseInt(body.bet, 10);
    const mineCount = parseInt(body.mineCount, 10);

    if (!bet || bet < 1) return json({ error: 'Ongeldige inzet' }, 400);
    if (!mineCount || mineCount < 1 || mineCount > 24) return json({ error: 'Ongeldig aantal mijnen (1-24)' }, 400);

    const deducted = await deductCoins(String(user._id), bet);
    if (!deducted) return json({ error: 'Niet genoeg coins' }, 400);

    // Generate random mine positions
    const positions = new Set();
    while (positions.size < mineCount) {
      positions.add(Math.floor(Math.random() * 25));
    }

    const gameId = createSession(String(user._id), bet, mineCount, positions);

    return json({ gameId, multiplier: calcMultiplier(mineCount, 0) });
  } catch (e) {
    return json({ error: e?.message }, 500);
  }
}

function json(data, status = 200) {
  return new Response(JSON.stringify(data), {
    status,
    headers: { 'Content-Type': 'application/json' },
  });
}
```

■ payment-webhook.json.js

src/pages/api/payment-webhook.json.js

```
// Placeholder webhook endpoint. Mollie will POST payment updates server-side.
// When implementing the real Mollie integration, validate signatures and
// update the user's coin balance based on the returned paymentId/session.

import dotenv from 'dotenv';
import fetch from 'node-fetch';
import { getPaymentByMollieId, markPaymentPaid, creditCoins } from '../../server/db.js';

dotenv.config();
const MOLLIE_API_KEY = process.env.MOLLIE_API_KEY || process.env.VITE_MOLLIE_API_KEY;

export async function POST({ request }) {
  try {
    const body = await request.json();
    console.log('payment-webhook received', body);

    const mollieId = body.id || (body.resourceId || body.paymentId || (body.payment ? body.payment.id : null));
    if(!mollieId) return new Response(JSON.stringify({ error: 'no id in webhook payload' }), { status: 400 });

    if(!MOLLIE_API_KEY) return new Response(JSON.stringify({ error: 'no mollie key configured' }), { status: 400 });

    const res = await fetch(`https://api.mollie.com/v2/payments/${mollieId}`, { headers: { Authorization: `Bearer ${MOLLIE_API_KEY}` } });
    const j = await res.json();
    const status = j.status;

    const payment = await getPaymentByMollieId(mollieId);
    if(!payment) return new Response(JSON.stringify({ error: 'payment not found' }), { status: 404 });

    if(status === 'paid'){
      if(payment.userId){
        const mapping = { p1:60, p5:120, p10:180 };
        const coins = mapping[payment.packageId] || 0;
        await creditCoins(payment.userId, coins);
      }
      await markPaymentPaid(mollieId);
    }

    return new Response(JSON.stringify({ ok:true, status }), { status: 200 });
  } catch(e) {
    console.error('payment-webhook.err', e);
    return new Response(JSON.stringify({ error: e.message }), { status: 500 });
  }
}
```

■ rules.json.js

src/pages/api/rules.json.js

```
const WIKI_API = 'https://nl.wikipedia.org/w/api.php';

const GAMES = {
  blackjack:      { title: 'Blackjack',      wikiPage: 'Blackjack' },
  roulette:       { title: 'Roulette',      wikiPage: 'Roulette_(spel)' },
  mines:          { title: 'Mines',         wikiPage: 'Mijnenveger' },
  poker:          { title: 'Texas Hold'em Poker', wikiPage: 'Texas Hold 'em' },
};

// In-memory cache: wikiPage → { data, expires }
const cache = new Map();
const TTL_MS = 60 * 60 * 1000; // 1 hour

async function fetchWiki(wikiPage) {
  const cached = cache.get(wikiPage);
  if (cached && Date.now() < cached.expires) return cached.data;

  const params = new URLSearchParams({
    action:      'query',
    titles:      wikiPage,
    prop:        'extracts|pageimages|info',
    explaintext: 'true',
    piprop:      'thumbnail',
    pithumbsize: '240',
    inprop:      'url',
    format:      'json',
    origin:      '*',
  });

  const res = await fetch(`${WIKI_API}?${params}`, {
    headers: { 'User-Agent': 'CMD-Casino/1.0 (school project; nl.wikipedia.org)' },
  });

  if (!res.ok) return null;

  const d = await res.json();
  const page = Object.values(d.query.pages)[0];
  if (page.missing !== undefined) return null;

  const data = {
    wikiTitle: page.title,
    sections:  parseExtract(page.extract),
    thumbnail: page.thumbnail?.source ?? null,
    sourceUrl: page.fullurl ?? `https://nl.wikipedia.org/wiki/${encodeURIComponent(wikiPage)}`,
  };

  cache.set(wikiPage, { data, expires: Date.now() + TTL_MS });
  return data;
}

// Split plain-text extract into [{title, text, level}] sections
function parseExtract(extract) {
  if (!extract) return [];

  // ==+ captures both == and === headers; \1 backreferences the opening ==
  const parts = extract.split(/\n+(==+)\s*(.+?)\s*\1\n/);
  const sections = [];

  // parts[0] is intro text, then [level-markers, title, text, level-markers, title, text, ...]
```

```

    if (parts[0].trim()) {
      sections.push({ title: null, level: 1, text: parts[0].trim() });
    }

    for (let i = 1; i < parts.length; i += 3) {
      const level = parts[i].length; // 2 = main section, 3 = subsection
      const title = parts[i + 1];
      const text = (parts[i + 2] ?? '').trim();
      if (title && text) sections.push({ title, level, text });
    }

    return sections;
  }

export async function GET({ url }) {
  const slug = url.searchParams.get('game');

  if (slug) {
    const game = GAMES[slug];
    if (!game) {
      return json({ error: 'Spel niet gevonden', available: Object.keys(GAMES) }, 404);
    }

    const wiki = await fetchWiki(game.wikiPage);
    if (!wiki) {
      return json({ error: 'Wikipedia niet bereikbaar' }, 502);
    }

    return json({ slug, title: game.title, ...wiki });
  }

  const list = Object.entries(GAMES).map(([slug, { title, wikiPage }]) => ({
    slug, title, wikiPage,
    endpoint: `/api/rules.json?game=${slug}`,
  }));

  return json({ games: list });
}

function json(data, status = 200) {
  return new Response(JSON.stringify(data), {
    status,
    headers: {
      'Content-Type': 'application/json',
      'Cache-Control': 'no-cache',
    },
  });
}

```

■ score.json.js

src/pages/api/score.json.js

```
import { saveScore, getScore } from '../../../server/db.js';
import { getSession, getTokenFromRequest } from '../../../lib/auth.js';

export async function POST({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if(!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });
    const body = await request.json();
    const score = Number(body.score) || 0;
    const userSub = String(user._id);
    await saveScore(userSub, score);
    return new Response(JSON.stringify({ ok:true }), { status: 200 });
  } catch(e) {
    console.error('score.post err', e && e.message);
    return new Response(JSON.stringify({ error: e && e.message }), { status: 500 });
  }
}

export async function GET({ request }) {
  try {
    const token = getTokenFromRequest(request);
    const user = await getSession(token);
    if(!user) return new Response(JSON.stringify({ error: 'unauthorized' }), { status: 401 });
    const userSub = String(user._id);
    const r = await getScore(userSub);
    return new Response(JSON.stringify({ score: r ? r.score : 0, updated_at: r ? r.updated_at : null }), { status: 200 });
  } catch(e) {
    console.error('score.get err', e && e.message);
    return new Response(JSON.stringify({ error: e && e.message }), { status: 500 });
  }
}
```

■ script.js

src/pages/api/script.js

```
// const API = "http://localhost:3000";

// async function signup() {
//   const username = document.getElementById("signup-username").value;
//   const password = document.getElementById("signup-password").value;

//   const res = await fetch(`${API}/signup`, {
//     method: "POST",
//     headers: {
//       "Content-Type": "application/json"
//     },
//     body: JSON.stringify({ username, password })
//   });

//   const data = await res.json();
//   document.getElementById("message").innerText = data.msg;
// }

// async function login() {
//   const username = document.getElementById("login-username").value;
//   const password = document.getElementById("login-password").value;

//   const res = await fetch(`${API}/login`, {
//     method: "POST",
//     headers: {
//       "Content-Type": "application/json"
//     },
//     body: JSON.stringify({ username, password })
//   });

//   const data = await res.json();
//   document.getElementById("message").innerText = data.msg;
// }
```

■ signup.js

src/pages/api/signup.js

```
import bcrypt from 'bcryptjs';
import { getDB } from '../../../server/mongodb.js';
import { createSession } from '../../../lib/auth.js';

export async function POST({ request }) {
  try {
    const { username, password } = await request.json();
    if (!username || !password)
      return new Response(JSON.stringify({ message: 'Gebruikersnaam en wachtwoord zijn verplicht' })), { status: 400 };
    if (username.length < 3 || username.length > 20)
      return new Response(JSON.stringify({ message: 'Gebruikersnaam moet 3-20 tekens zijn' })), { status: 400 };
    if (!/^[a-zA-Z0-9_]+$/.test(username))
      return new Response(JSON.stringify({ message: 'Gebruikersnaam mag alleen letters, cijfers en _ bevatten' })), { status: 400 };
    if (password.length < 6)
      return new Response(JSON.stringify({ message: 'Wachtwoord moet minimaal 6 tekens zijn' })), { status: 400 };

    const db = await getDB();
    const existing = await db.collection('users').findOne({ username });
    if (existing)
      return new Response(JSON.stringify({ message: 'Gebruikersnaam al in gebruik' })), { status: 409 };

    const hash = await bcrypt.hash(password, 10);
    const result = await db.collection('users').insertOne({
      username,
      password: hash,
      coins_amount: 60,
      lastDailyReward: null,
      createdAt: new Date()
    });

    const token = createSession({ _id: result.insertedId, username });
    return new Response(JSON.stringify({ message: 'Account aangemaakt', token }), {
      status: 201,
      headers: { 'Set-Cookie': `session=${token}; Path=/; HttpOnly; SameSite=Lax; Max-Age=604800` }
    });
  } catch (e) {
    return new Response(JSON.stringify({ message: 'Server fout: ' + e?.message })), { status: 500 };
  }
}
```

```

export const prerender = false;

// Module-level cache persists for the lifetime of the server process
const cache = new Map();

export async function GET({ request }) {
  const { searchParams } = new URL(request.url);
  const text = searchParams.get('text');

  if (!text || text.length > 200) {
    return new Response('Missing or invalid text', { status: 400 });
  }

  const apiKey = import.meta.env.ELEVENLABS_API_KEY;
  const voiceId = import.meta.env.ELEVENLABS_VOICE_ID || 'pNInz6obpgDQGcFmaJgB'; // Adam

  if (!apiKey) {
    return new Response('ElevenLabs not configured', { status: 503 });
  }

  if (cache.has(text)) {
    return new Response(cache.get(text), {
      headers: {
        'Content-Type' : 'audio/mpeg',
        'Cache-Control' : 'public, max-age=86400',
      },
    });
  }

  let elevRes;
  try {
    elevRes = await fetch(
      `https://api.elevenlabs.io/v1/text-to-speech/${voiceId}`,
      {
        method : 'POST',
        headers : {
          'xi-api-key' : apiKey,
          'Content-Type' : 'application/json',
          'Accept' : 'audio/mpeg',
        },
        body: JSON.stringify({
          text,
          model_id : 'eleven_multilingual_v2',
          voice_settings : { stability: 0.45, similarity_boost: 0.8 },
        }),
      }
    );
  } catch (e) {
    console.error('[TTS] fetch error:', e);
    return new Response('TTS request failed', { status: 502 });
  }

  if (!elevRes.ok) {
    const body = await elevRes.text().catch(() => '');
    console.error('[TTS] ElevenLabs error', elevRes.status, body);
    return new Response('TTS upstream error', { status: 502 });
  }

  const audio = await elevRes.arrayBuffer();

```

```
cache.set(text, audio);

return new Response(audio, {
  headers: {
    'Content-Type' : 'audio/mpeg',
    'Cache-Control' : 'public, max-age=86400',
  },
});
}
```

■ webhook.js

src/pages/api/webhook.js

```
import { createMollieClient } from '@mollie/api-client';
import { getPaymentByMollieId, markPaymentPaid, creditCoins } from '../../../server/db.js';

export async function POST({ request }) {
  try {
    // Mollie stuurt de webhook als form-encoded body: id=tr_XXXXX
    const body = await request.text();
    const params = new URLSearchParams(body);
    const id = params.get('id');

    if (!id) {
      console.error('webhook: geen id ontvangen');
      return new Response('ok', { status: 200 });
    }

    // Controleer de betaalstatus rechtstreeks bij Mollie
    const mollie = createMollieClient({ apiKey: import.meta.env.MOLLIE_API_KEY });
    const molliePayment = await mollie.payments.get(id);

    if (molliePayment.status !== 'paid') {
      // Nog niet betaald (bijv. pending of cancelled) – niets doen
      return new Response('ok', { status: 200 });
    }

    // Zoek de bijbehorende betaling in onze database
    const existing = await getPaymentByMollieId(id);

    if (!existing) {
      console.error('webhook: betaling niet gevonden in database voor id', id);
      return new Response('ok', { status: 200 });
    }

    if (existing.status === 'paid') {
      // Al verwerkt, voorkom dubbele bijschrijving
      return new Response('ok', { status: 200 });
    }

    // Schrijf de coins bij en markeer als betaald
    await markPaymentPaid(id);
    await creditCoins(existing.userId, existing.coins);

    console.log('webhook: betaling verwerkt', id, '- coins bijgeschreven:', existing.coins, 'voor gebruiker', existing.userId);

    return new Response('ok', { status: 200 });
  } catch (e) {
    console.error('webhook fout:', e.message);
    return new Response('ok', { status: 200 });
  }
}
```

■ .!98333!multiplayer.js

src/scripts/.!98333!multiplayer.js

```
const token = localStorage.getItem('token');
if (!token) { location.href = '/login'; }

const params = new URLSearchParams(location.search);
const roomId = params.get('room');
if (!roomId || !/^d+$/.test(roomId)) { location.href = '/'; }
```

■ .!98340!blackjack.js

src/scripts/.!98340!blackjack.js

```
// Minimal single-player Blackjack vs dealer
const ranks = ['A','2','3','4','5','6','7','8','9','10','J','Q','K'];
const suits = ['♠','♥','♦','♣'];

function createDeck(){
  const d = [];
  for(const s of suits) for(const r of ranks) d.push({rank:r,suit:s});
  return d;
}

function shuffle(deck){
  for(let i=deck.length-1;i>0;i--){
    const j = Math.floor(Math.random()*(i+1));
    [deck[i],deck[j]] = [deck[j],deck[i]];
  }
}

function cardValue(card){
  if(!card || typeof card.rank !== 'string') return 0;
  if(card.rank === 'A') return 11;
  if(['J','Q','K'].includes(card.rank)) return 10;
  const v = parseInt(card.rank,10);
  return Number.isNaN(v) ? 0 : v;
}

function handValue(hand){
  let total=0, aces=0;
  for(const c of hand){
    if(c.rank === 'A'){ aces++; total += 11; } else total += cardValue(c);
  }
  while(total>21 && aces>0){ total -=10; aces--; }
  return total;
}

function isRed(card){ return card.suit === '♥' || card.suit === '♦'; }

function makeCardInner(card){
  return `<div class="rank">${card.rank}</div><div class="suit-center">${card.suit}</div>`;
}

function makeCardEl(card, hidden = false){
  const el = document.createElement('div');
  if(hidden){
    el.className = 'card back';
  } else {
    el.className = 'card' + (isRed(card) ? ' red' : '');
    el.innerHTML = makeCardInner(card);
  }
  return el;
}

function delay(ms){ return new Promise(r => setTimeout(r, ms)); }

let deck = [];
let dealer = { hand: [] };
let player = { hand: [] };
let bank = 0;
let bet = 50;
```

```

let pendingSave = null;
let holeRevealed = false;

// Split / double state
let isSplit = false;
let splitHandArr = [];
let splitBet = 0;
let activeSide = 'main'; // 'main' | 'split'

function getToken(){
  try { return localStorage.getItem('token'); } catch(e){ return null; }
}

async function loadBalance(){
  try{
    const token = getToken();
    if(!token) return;
    const res = await fetch('/api/balance.json', { method: 'GET', headers: { 'Authorization': 'Bearer ' + token } });
    if(!res.ok) return;
    const j = await res.json();
    if(j && typeof j.balance === 'number'){
      bank = j.balance;
      if(bankEl) bankEl.textContent = bank.toLocaleString('nl-NL');
      if(dealBtn) dealBtn.disabled = false;
    }
  } catch(e){ console.warn('loadBalance failed', e); }
}

function saveBalance(){
  const token = getToken();
  if(!token) return Promise.resolve();
  pendingSave = window.pendingSave = fetch('/api/balance.json', {
    method: 'POST',
    credentials: 'include',
    headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + token },
    body: JSON.stringify({ balance: bank })
  }).then(r => {
    if(!r.ok) console.warn('saveBalance HTTP error', r.status);
  }).catch(e => console.warn('saveBalance failed', e));
  return pendingSave;
}

const dealerHandEl = document.getElementById('dealerHand');
const playerHandEl = document.getElementById('playerHand');
const splitHandEl = document.getElementById('splitHand');
const splitHandWrap = document.getElementById('splitHandWrap');
const playerHandCol = document.getElementById('playerHandCol');
const dealerScoreEl = document.getElementById('dealerScore');
const playerScoreEl = document.getElementById('playerScore');
const splitScoreEl = document.getElementById('splitScore');
const bankEl = document.getElementById('bank');
const messageEl = document.getElementById('message');
const dealerBadge = document.getElementById('dealerScoreBadge');
const playerBadge = document.getElementById('playerScoreBadge');
const splitBadge = document.getElementById('splitScoreBadge');

const dealBtn = document.getElementById('dealBtn');
const hitBtn = document.getElementById('hitBtn');
const standBtn = document.getElementById('standBtn');
const doubleBtn = document.getElementById('doubleBtn');
const splitBtn = document.getElementById('splitBtn');
const newBtn = document.getElementById('newBtn');

```

```

const betInput = document.getElementById('betAmount');

function fatal(msg){
  console.error('Blackjack fatal:', msg);
  if(messageEl) messageEl.textContent = 'Error: ' + msg;
  if(dealBtn) dealBtn.disabled = true;
  if(hitBtn) hitBtn.disabled = true;
  if(standBtn) standBtn.disabled = true;
  throw new Error(msg);
}

if(!dealerHandEl || !playerHandEl || !dealerScoreEl || !playerScoreEl || !bankEl || !messageEl){
  fatal('Missing required DOM elements.');
```

```

}
if(!dealBtn || !hitBtn || !standBtn || !newBtn || !betInput){
  fatal('Missing control elements.');
```

```

}
```

account.js

src/scripts/account.js

```
async function loadAccount() {
  const token = sessionStorage.getItem('token');
  if (!token) { document.getElementById('not-logged').style.display = 'block'; return; }
  const res = await fetch('/api/me', { headers: { Authorization: 'Bearer ' + token }, credentials: 'include' });
  if (!res.ok) { sessionStorage.removeItem('token'); document.getElementById('not-logged').style.display = 'block'; }
  const me = await res.json();
  document.getElementById('user-name').textContent = me.username;
  document.getElementById('user-coins').textContent = (me.coins_amount ?? 0).toLocaleString('nl-NL');
  document.getElementById('user-created').textContent = me.createdAt ? new Date(me.createdAt).toLocaleDateString() : '';
  const avatarEl = document.getElementById('avatar-initials');
  if (me.username.toLowerCase().includes('cyd')) {
    avatarEl.innerHTML = '';
  } else {
    avatarEl.textContent = me.username.slice(0, 2).toUpperCase();
  }
  document.getElementById('account-card').style.display = 'block';
}

document.getElementById('reward-btn')?.addEventListener('click', async () => {
  const token = sessionStorage.getItem('token');
  const msg = document.getElementById('reward-msg');
  try {
    const res = await fetch('/api/daily-reward', { method: 'POST', headers: { Authorization: 'Bearer ' + token } });
    const data = await res.json();
    if (data.claimed) {
      msg.className = 'ok'; msg.textContent = data.message;
      loadAccount();
    } else {
      const next = data.nextClaimAt ? ' Morgen weer: ' + new Date(data.nextClaimAt).toLocaleTimeString('nl-NL') : '';
      msg.className = 'err'; msg.textContent = data.message + next;
    }
  } catch {
    msg.className = 'err'; msg.textContent = 'Fout bij claimen';
  }
});

document.getElementById('logout-btn')?.addEventListener('click', () => {
  sessionStorage.removeItem('token');
  document.cookie = 'session=; Path=/; Max-Age=0';
  location.href = '/login';
});

loadAccount();

window.addEventListener('pageshow', (e) => { if (e.persisted) loadAccount(); });
```

■ auth-client.js

src/scripts/auth-client.js

```
// // lightweight local auth shim: read token from localStorage
// export async function initAuth(){ return true; }
// export async function getAuthToken(){
//   try{ return localStorage.getItem('token'); }catch(e){ return null; }
// }

// // expose helper on window for legacy code
// if(typeof window !== 'undefined'){ window.getAuthToken = async ()=>{ return localStorage.getItem('token') }
```

■ auth-score.js

src/scripts/auth-score.js

```
// helper to save and load authenticated user score
// export async function saveScoreToServer(token, score){
//   const t = token || (typeof localStorage !== 'undefined' ? localStorage.getItem('token') : null);
//   const res = await fetch('/api/score.json', { method: 'POST', headers: { 'Content-Type': 'application/json' },
//   return res.json();
// }

// export async function loadScoreFromServer(token){
//   const t = token || (typeof localStorage !== 'undefined' ? localStorage.getItem('token') : null);
//   const res = await fetch('/api/score.json', { method: 'GET', headers: { 'Authorization': 'Bearer ' + t } });
//   return res.json();
// }
```

■ blackjack.js

src/scripts/blackjack.js

```
// Minimal single-player Blackjack vs dealer
const ranks = ['A','2','3','4','5','6','7','8','9','10','J','Q','K'];
const suits = ['♠','♥','♦','♣'];

function createDeck(){
  const d = [];
  for(const s of suits) for(const r of ranks) d.push({rank:r,suit:s});
  return d;
}

function shuffle(deck){
  for(let i=deck.length-1;i>0;i--){
    const j = Math.floor(Math.random()*(i+1));
    [deck[i],deck[j]] = [deck[j],deck[i]];
  }
}

function cardValue(card){
  if(!card || typeof card.rank !== 'string') return 0;
  if(card.rank === 'A') return 11;
  if(['J','Q','K'].includes(card.rank)) return 10;
  const v = parseInt(card.rank,10);
  return Number.isNaN(v) ? 0 : v;
}

function handValue(hand){
  let total=0, aces=0;
  for(const c of hand){
    if(c.rank === 'A'){ aces++; total += 11; } else total += cardValue(c);
  }
  while(total>21 && aces>0){ total -=10; aces--; }
  return total;
}

function isRed(card){ return card.suit === '♥' || card.suit === '♦'; }

function makeCardInner(card){
  return `<div class="rank">${card.rank}</div><div class="suit-center">${card.suit}</div>`;
}

function makeCardEl(card, hidden = false){
  const el = document.createElement('div');
  if(hidden){
    el.className = 'card back';
  } else {
    el.className = 'card' + (isRed(card) ? ' red' : '');
    el.innerHTML = makeCardInner(card);
  }
  return el;
}

function delay(ms){ return new Promise(r => setTimeout(r, ms)); }

let deck = [];
let dealer = { hand: [] };
let player = { hand: [] };
let bank = 0;
let bet = 50;
```

```

let originalBet = 50;
let pendingSave = null;
let holeRevealed = false;

// Split / double state
let isSplit = false;
let splitHandArr = [];
let splitBet = 0;
let activeSide = 'main'; // 'main' | 'split'

function getToken(){
  try { return sessionStorage.getItem('token'); } catch(e){ return null; }
}

async function loadBalance(){
  try{
    const token = getToken();
    if(!token) return;
    const res = await fetch('/api/balance.json', { method: 'GET', headers: { 'Authorization': 'Bearer ' + token } });
    if(!res.ok) return;
    const j = await res.json();
    if(j && typeof j.balance === 'number'){
      bank = j.balance;
      if(bankEl) bankEl.textContent = bank.toLocaleString('nl-NL');
      if(dealBtn) dealBtn.disabled = false;
    }
  } catch(e){ console.warn('loadBalance failed', e); }
}

function saveBalance(){
  const token = getToken();
  if(!token) return Promise.resolve();
  pendingSave = window.pendingSave = fetch('/api/balance.json', {
    method: 'POST',
    credentials: 'include',
    headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + token },
    body: JSON.stringify({ balance: bank })
  }).then(r => {
    if(!r.ok) console.warn('saveBalance HTTP error', r.status);
  }).catch(e => console.warn('saveBalance failed', e));
  return pendingSave;
}

const dealerHandEl = document.getElementById('dealerHand');
const playerHandEl = document.getElementById('playerHand');
const splitHandEl = document.getElementById('splitHand');
const splitHandWrap = document.getElementById('splitHandWrap');
const playerHandCol = document.getElementById('playerHandCol');
const dealerScoreEl = document.getElementById('dealerScore');
const playerScoreEl = document.getElementById('playerScore');
const splitScoreEl = document.getElementById('splitScore');
const bankEl = document.getElementById('bank');
const messageEl = document.getElementById('message');
const dealerBadge = document.getElementById('dealerScoreBadge');
const playerBadge = document.getElementById('playerScoreBadge');
const splitBadge = document.getElementById('splitScoreBadge');

const dealBtn = document.getElementById('dealBtn');
const hitBtn = document.getElementById('hitBtn');
const standBtn = document.getElementById('standBtn');
const doubleBtn = document.getElementById('doubleBtn');
const splitBtn = document.getElementById('splitBtn');

```



```

        const el = makeCardEl(splitHandArr[i]);
        el.style.animationDelay = '0ms';
        splitHandEl.appendChild(el);
    }
}

function setScore(el, badge, value, opts = {}){
    el.textContent = String(value);
    if(!badge) return;
    badge.className = 'score-badge';
    if(opts.bust) badge.classList.add('bust');
    else if(opts.bj) badge.classList.add('blackjack');
    else if(opts.good) badge.classList.add('good');
}

function showOutcome(text, type = 'push', voiceFn = null){
    if(!messageEl) return;
    messageEl.className = 'outcome-msg ' + type;
    messageEl.textContent = text;
    void messageEl.offsetWidth;
    messageEl.classList.add('visible');

    if(window.Sound){
        if(type === 'bj') { Sound.playWin(); Sound.say.blackjack(); }
        else if(type === 'win') { Sound.playWin(); (voiceFn || Sound.say.playerWins()); }
        else if(type === 'lose') { Sound.playLose(); (voiceFn || Sound.say.dealerWins()); }
        else { Sound.say.push(); }
    }
}

function hideOutcome(){
    if(!messageEl) return;
    messageEl.classList.remove('visible');
}

function flashCards(handEl, type){
    Array.from(handEl.children).forEach(c => {
        c.classList.remove('card-win', 'card-lose');
        void c.offsetWidth;
        c.classList.add(type === 'win' ? 'card-win' : 'card-lose');
    });
}

function updateActionBtns(phase){
    const playing = phase === 'playing';
    hitBtn.disabled = !playing;
    standBtn.disabled = !playing;

    if(playing){
        const activeHand = activeSide === 'main' ? player.hand : splitHandArr;
        const activeBetAmt = activeSide === 'main' ? bet : splitBet;
        doubleBtn.disabled = activeHand.length !== 2 || bank < activeBetAmt;
        splitBtn.disabled = isSplit || activeSide !== 'main' ||
            player.hand.length !== 2 ||
            player.hand[0].rank !== player.hand[1].rank ||
            bank < bet;
    } else {
        doubleBtn.disabled = true;
        splitBtn.disabled = true;
    }
}

```



```

const pCards = playerHandEl.querySelectorAll('.card');
const dCards = dealerHandEl.querySelectorAll('.card');
if(pCards[0]) pCards[0].style.animationDelay = '0ms';
if(dCards[0]) dCards[0].style.animationDelay = '180ms';
if(pCards[1]) pCards[1].style.animationDelay = '360ms';
if(dCards[1]) dCards[1].style.animationDelay = '540ms';

const pval = handValue(player.hand);
setScore(playerScoreEl, playerBadge, pval, { bj: pval === 21 });
if(pval === 21){
    finishHand();
} else {
    updateActionBtns('playing');
}
}

function playerHit(){
    const activeHand = activeSide === 'main' ? player.hand : splitHandArr;
    const valBefore = handValue(activeHand);
    if(window.Sound){
        Sound.playCard();
        if(valBefore >= 20)      setTimeout(() => Sound.say.boldMove(), 150);
        else if(valBefore >= 18) setTimeout(() => Sound.say.interesting(), 150);
    }
    activeHand.push(deck.pop());

    if(activeSide === 'main'){
        renderHands(true);
        const pv = handValue(player.hand);
        setScore(playerScoreEl, playerBadge, pv, { bust: pv > 21 });
        if(pv > 21) finishHand(); else updateActionBtns('playing');
    } else {
        renderSplitHand();
        const sv = handValue(splitHandArr);
        setScore(splitScoreEl, splitBadge, sv, { bust: sv > 21 });
        if(sv > 21) finishHand(); else updateActionBtns('playing');
    }
}

function doDouble(){
    if(window.Sound){ Sound.playChip(); Sound.say.doubling(); setTimeout(() => Sound.playCard(), 150); }
    const activeBetAmt = activeSide === 'main' ? bet : splitBet;
    bank -= activeBetAmt;
    if(activeSide === 'main') bet *= 2; else splitBet *= 2;
    bankEl.textContent = bank.toLocaleString('nl-NL');

    const activeHand = activeSide === 'main' ? player.hand : splitHandArr;
    activeHand.push(deck.pop());

    if(activeSide === 'main'){
        renderHands(true);
        const pv = handValue(player.hand);
        setScore(playerScoreEl, playerBadge, pv, { bust: pv > 21 });
    } else {
        renderSplitHand();
        const sv = handValue(splitHandArr);
        setScore(splitScoreEl, splitBadge, sv, { bust: sv > 21 });
    }
    finishHand();
}

function doSplit(){

```

```

    if(window.Sound){ Sound.playChip(); Sound.say.split(); setTimeout(() => Sound.playCard(), 100); setTimeout(
    const secondCard = player.hand.pop();
    splitHandArr = [secondCard];
    splitBet = bet;
    bank -= bet;
    isSplit = true;
    activeSide = 'main';

    // Deal one new card to each hand
    player.hand.push(deck.pop());
    splitHandArr.push(deck.pop());

    // Show split area
    if(splitHandWrap) splitHandWrap.style.display = '';

    // Re-render both hands from scratch
    playerHandEl.innerHTML = '';
    renderHands(true);
    if(splitHandEl) splitHandEl.innerHTML = '';
    renderSplitHand();

    setScore(playerScoreEl, playerBadge, handValue(player.hand));
    setScore(splitScoreEl, splitBadge, handValue(splitHandArr));
    bankEl.textContent = bank.toLocaleString('nl-NL');

    updateHandHighlight();
    updateActionBtns('playing');
}

async function dealerPlay(){
    if(window.Sound) Sound.playSwoosh();
    renderHands(false);
    let dv = handValue(dealer.hand);
    setScore(dealerScoreEl, dealerBadge, dv);

    await delay(480);

    while(dv < 17 && deck.length > 0){
        const card = deck.pop();
        if(!card) break;
        dealer.hand.push(card);
        dv = handValue(dealer.hand);
        if(window.Sound) Sound.playCard();
        renderHands(false);
        setScore(dealerScoreEl, dealerBadge, dv, { bust: dv > 21 });
        await delay(420);
    }
}

function resolveHand(pv, dv, handLen, handEl, currentBet, allowBJ = true){
    let payout = 0;
    let outcome = '';
    let type = 'push';

    if(pv > 21){
        outcome = 'Bust – verloren';
        type = 'lose';
        flashCards(handEl, 'lose');
    } else if(dv === 21 && dealer.hand.length === 2){
        outcome = 'Dealer blackjack – verloren!';
        type = 'lose';
    } else if(dv > 21){

```

```

        outcome = 'Dealer bust – gewonnen!';
        type = 'win';
        payout = currentBet * 2;
        flashCards(handEl, 'win');
    } else if(pv === dv){
        outcome = 'Gelijkspel';
        type = 'push';
        payout = currentBet;
    } else if(allowBJ && pv === 21 && handLen === 2){
        outcome = 'Blackjack! +1.5x';
        type = 'bj';
        payout = Math.floor(currentBet * 2.5);
        flashCards(handEl, 'win');
    } else if(pv > dv){
        outcome = 'Je wint!';
        type = 'win';
        payout = currentBet * 2;
        flashCards(handEl, 'win');
    } else {
        outcome = 'Dealer wint';
        type = 'lose';
        flashCards(handEl, 'lose');
    }
}

return { payout, outcome, type };
}

async function finishSplitAndDealer(){
    await dealerPlay();

    const dv = handValue(dealer.hand);
    setScore(dealerScoreEl, dealerBadge, dv, { bust: dv > 21 });

    if(isSplit){
        const pv = handValue(player.hand);
        const sv = handValue(splitHandArr);
        setScore(playerScoreEl, playerBadge, pv, { bust: pv > 21 });
        setScore(splitScoreEl, splitBadge, sv, { bust: sv > 21 });

        // Split hands don't get BJ bonus (standard casino rule)
        const r1 = resolveHand(pv, dv, player.hand.length, playerHandEl, bet, false);
        const r2 = resolveHand(sv, dv, splitHandArr.length, splitHandEl, splitBet, false);

        bank += r1.payout + r2.payout;

        let combinedOutcome;
        if(r1.type === r2.type){
            const labels = { win: 'Beide gewonnen!', lose: 'Beide verloren', push: 'Beide gelijkspel' };
            combinedOutcome = labels[r1.type] || `${r1.outcome} | ${r2.outcome}`;
        } else {
            combinedOutcome = `Hand 1: ${r1.outcome} | Hand 2: ${r2.outcome}`;
        }

        const totalReturn = r1.payout + r2.payout;
        const totalStake = bet + splitBet;
        const overallType = totalReturn > totalStake ? 'win' : totalReturn === totalStake ? 'push' : 'lose';
        if(overallType === 'win' && window.Sound) Sound.playCashRegister();
        let splitVoiceFn = null;
        if(window.Sound){
            if(bank === 0 && overallType === 'lose') splitVoiceFn = Sound.say.houseWins;
            else if(totalReturn >= originalBet * 4) splitVoiceFn = Sound.say.bigWinner;
            else if(dv > 21 && overallType === 'win') splitVoiceFn = Sound.say.dealerBust;
        }
    }
}

```

[illegible]

```
if(dealBtn)    dealBtn.addEventListener('click', () => { if(window.Sound) Sound.playShuffle(); resetTable();
if(hitBtn)     hitBtn.addEventListener('click', () => { playerHit(); });
if(standBtn)   standBtn.addEventListener('click', () => { if(window.Sound) Sound.say.standing(); finishHand();
if(doubleBtn)  doubleBtn.addEventListener('click', () => { doDouble(); });
if(splitBtn)   splitBtn.addEventListener('click', () => { doSplit(); });
if(newBtn)     newBtn.addEventListener('click', () => { if(window.Sound){ Sound.playShuffle(); setTimeout(
```

■ client.js

src/scripts/client.js

```
const token = sessionStorage.getItem('token');
const proto = location.protocol === 'https:' ? 'wss' : 'ws';
const socket = new WebSocket(proto + '://' + location.host + '/ws');

socket.addEventListener('open', () => {
  console.log('verbonden!');
  socket.send(JSON.stringify({ type: 'PING' }));
});

socket.addEventListener('message', (event) => {
  const msg = JSON.parse(event.data);

  switch (msg.type){
    case 'TABLE_LIST': renderLobby(msg.tables); break;
    case 'GAME_STATE': renderTable(msg.state); break;
    case 'CARD_DEALT': renderCard(msg.card); break;
    case 'ERROR': showError(msg.message); break;
    case 'PONG': console.log('round trip ms:', Date.now() - msg.time); break;
  }
});

function sendAction(type, payload = {}){
  socket.send(JSON.stringify({type, ...payload}));
}
```

index.js

src/scripts/index.js

```
function loadUser() {
  const token = sessionStorage.getItem('token');
  if (!token) {
    const notLogged = document.getElementById('not-logged');
    const loggedIn = document.getElementById('logged-in');
    if (notLogged) notLogged.style.display = 'block';
    if (loggedIn) loggedIn.style.display = 'none';
    return;
  }
  fetch('/api/me', { headers: { Authorization: 'Bearer ' + token } })
    .then(r => r.ok ? r.json() : Promise.reject())
    .then(d => {
      const usernameEl = document.getElementById('username');
      const ECPointsEl = document.getElementById('EC-Punten');
      const loggedIn = document.getElementById('logged-in');
      if (usernameEl) usernameEl.textContent = d.username;
      if (ECPointsEl) ECPointsEl.textContent = (d.coins_amount ?? 0).toLocaleString('nl-NL');
      if (loggedIn) loggedIn.style.display = 'block';
    })
    .catch(() => { sessionStorage.removeItem('token'); location.reload(); });
}

loadUser();

window.addEventListener('pageshow', (e) => { if (e.persisted) loadUser(); });

{
  const dailyMsg = sessionStorage.getItem('dailyReward');
  if (dailyMsg) {
    const el = document.getElementById('daily-msg');
    if (el) { el.textContent = dailyMsg; el.style.display = 'block'; }
    sessionStorage.removeItem('dailyReward');
  }
}

function goToMultiplayer() {
  const roomInput = document.getElementById('roomInput');
  const msg = document.getElementById('msg');
  const room = roomInput?.value.trim();
  if (!room || !/^\d+$/ .test(room)) { msg.textContent = 'Gebruik alleen cijfers voor het kamer-nummer.'; return; }
  msg.textContent = '';
  location.href = '/multiplayer?room=' + encodeURIComponent(room);
}

function goToPoker() {
  const roomInput = document.getElementById('roomInput');
  const msg = document.getElementById('msg');
  const room = roomInput?.value.trim();
  if (!room || !/^\d+$/ .test(room)) { msg.textContent = 'Gebruik alleen cijfers voor het kamer-nummer.'; return; }
  msg.textContent = '';
  location.href = '/poker?room=' + encodeURIComponent(room);
}

document.getElementById('soloBtn')?.addEventListener('click', () => { location.href = '/blackjack'; });
document.getElementById('rouletteBtn')?.addEventListener('click', () => { location.href = '/roulette'; });
document.getElementById('minesBtn')?.addEventListener('click', () => { location.href = '/mines'; });
document.getElementById('multiBtn')?.addEventListener('click', goToMultiplayer);
document.getElementById('pokerBtn')?.addEventListener('click', goToPoker);
```

```
document.getElementById('roomInput')?.addEventListener('keydown', (e) => { if (e.key === 'Enter') goToMulti
```

■ leaderboard.js

src/scripts/leaderboard.js

```
async function loadLeaderboard() {
  const token = sessionStorage.getItem('token');
  let myUsername = null;

  if (token) {
    try {
      const me = await fetch('/api/me', { headers: { Authorization: 'Bearer ' + token } });
      if (me.ok) myUsername = (await me.json()).username;
    } catch {}
  }

  try {
    const res = await fetch('/api/leaderboard.json');
    if (!res.ok) throw new Error();
    const { top } = await res.json();

    document.getElementById('loading').style.display = 'none';
    const list = document.getElementById('list');

    const medals = { 1: '■', 2: '■', 3: '■' };

    top.forEach((p, i) => {
      const row = document.createElement('div');
      row.className = 'row' + (p.rank <= 3 ? ' rank-' + p.rank : '');
      row.style.animationDelay = (i * 60) + 'ms';

      const rankBadge = document.createElement('div');
      rankBadge.className = 'rank-badge' + (medals[p.rank] ? ' medal' : '');
      rankBadge.textContent = medals[p.rank] ?? String(p.rank);

      const player = document.createElement('div');
      player.className = 'player';

      const avatar = document.createElement('div');
      avatar.className = 'avatar';
      if (p.username.toLowerCase().includes('cyd')) {
        const img = document.createElement('img');
        img.src = '/easter-egg.png';
        img.alt = p.username;
        img.style.cssText = 'width:100%;height:100%;object-fit:cover;border-radius:50%';
        avatar.appendChild(img);
      } else {
        avatar.textContent = p.username.slice(0, 2).toUpperCase();
      }

      const nameWrap = document.createElement('div');
      nameWrap.style.cssText = 'display:flex;align-items:center;gap:6px;min-width:0';

      const nameEl = document.createElement('span');
      nameEl.className = 'username' + (p.username === myUsername ? ' me' : '');
      nameEl.textContent = p.username;

      nameWrap.appendChild(nameEl);

      if (p.username === myUsername) {
        const badge = document.createElement('span');
        badge.className = 'you-badge';
        badge.textContent = 'JIJ';
      }
    });
  }
}
```

```

        namewrap.appendChild(badge);
    }

    player.appendChild(avatar);
    player.appendChild(namewrap);

    const coinsWrap = document.createElement('div');
    coinsWrap.className = 'coins-wrap';

    const coinIcon = document.createElement('span');
    coinIcon.className = 'coin-icon';
    coinIcon.textContent = '';

    const coinsEl = document.createElement('span');
    coinsEl.className = 'coins';
    coinsEl.textContent = p.coins_amount.toLocaleString('nl-NL');

    coinsWrap.appendChild(coinIcon);
    coinsWrap.appendChild(coinsEl);

    row.appendChild(rankBadge);
    row.appendChild(player);
    row.appendChild(coinsWrap);
    list.appendChild(row);
});

if (top.length === 0) {
    list.textContent = 'Nog geen spelers.';
    list.style.cssText = 'text-align:center;color:rgba(255,255,255,0.3);padding:32px 0;font-size:14px';
}
} catch {
    document.getElementById('loading').style.display = 'none';
    document.getElementById('error').style.display = 'block';
}
}

loadLeaderboard();

```

■ login.js

src/scripts/login.js

```
const btn = document.getElementById('btn');
const msg = document.getElementById('msg');

async function login() {
  if (!btn || !msg) return;
  btn.disabled = true; btn.textContent = 'Bezig...';
  msg.textContent = ''; msg.className = '';
  const username = document.getElementById('username').value.trim();
  const password = document.getElementById('password').value;
  try {
    const res = await fetch('/api/login', {
      method: 'POST', credentials: 'include',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });
    const data = await res.json();
    if (res.ok && data.token) {
      sessionStorage.setItem('token', data.token);
      if (data.dailyReward) sessionStorage.setItem('dailyReward', 'Dagelijkse beloning: +3 EC-Punten!');
      msg.className = 'ok'; msg.textContent = 'Ingelogd! Je wordt doorgestuurd...';
      setTimeout(() => { location.href = '/'; }, 600);
    } else {
      msg.className = 'err'; msg.textContent = data.message || 'Inloggen mislukt';
    }
  } catch {
    msg.className = 'err'; msg.textContent = 'Netwerkfout, probeer opnieuw';
  } finally {
    btn.disabled = false; btn.textContent = 'Inloggen';
  }
}

btn?.addEventListener('click', login);
document.addEventListener('keydown', e => { if (e.key === 'Enter') login(); });
```

■ logout.js

src/scripts/logout.js

```
document.getElementById('confirmBtn')?.addEventListener('click', () => {  
  sessionStorage.removeItem('token');  
  location.href = '/login';  
});
```


src/scripts/mines.js

```
const token = sessionStorage.getItem('token');
if (!token) { location.href = '/login'; }

// State
let gameId      = null;
let mineCount   = 3;
let currentBet  = 1;
let gameActive  = false;

// DOM refs
const bankEl    = document.getElementById('bank');
const minesGrid = document.getElementById('minesGrid');
const multiplierBar = document.getElementById('multiplierBar');
const multiplierValue = document.getElementById('multiplierValue');
const winPreview = document.getElementById('winPreview');
const betAmountEl = document.getElementById('betAmount');
const ctrlSetup = document.getElementById('ctrlSetup');
const ctrlGame = document.getElementById('ctrlGame');
const startBtn = document.getElementById('startBtn');
const cashoutBtn = document.getElementById('cashoutBtn');
const cashoutAmountEl = document.getElementById('cashoutAmount');
const gameMsg = document.getElementById('gameMsg');
const errorMsg = document.getElementById('errorMsg');

// Balance
async function loadBalance() {
  try {
    const r = await fetch('/api/balance.json', { headers: { Authorization: 'Bearer ' + token } });
    if (!r.ok) return;
    const d = await r.json();
    if (bankEl && typeof d.balance === 'number') bankEl.textContent = d.balance.toLocaleString('nl-NL');
  } catch {}
}

loadBalance();

// Grid
function buildGrid(interactive = false) {
  minesGrid.innerHTML = '';
  for (let i = 0; i < 25; i++) {
    const btn = document.createElement('button');
    btn.className = 'mine-tile';
    btn.dataset.index = i;
    btn.disabled = !interactive;
    btn.addEventListener('click', () => onTileClick(i, btn));
    minesGrid.appendChild(btn);
  }
}

function getTile(index) {
  return minesGrid.querySelector(`[data-index="${index}"]`);
}

function revealGem(btn) {
```

[illegible]

```

        loadBalance();
    } catch {
        showError('Verbindingsfout');
        startBtn.disabled = false;
    }
}

async function onTileClick(index, btn) {
    if (!gameActive || !gameId || btn.disabled) return;

    disableAllTiles();

    try {
        const d = await api('/api/mines/reveal.json', { gameId, index });
        if (d.error) { showError(d.error); enableUnrevealedTiles(); return; }

        if (d.mine) {
            revealMine(btn, true);
            d.minePositions?.forEach(pos => { if (pos !== index) revealMine(getTile(pos), false); });

            gameActive = false;
            if (gameMsg) gameMsg.textContent = '■ Boem! Verloren.';
            if (cashoutBtn) cashoutBtn.disabled = true;
            if (window.Sound) { Sound.playLose(); Sound.say.dealerWins(); }

            setTimeout(resetToSetup, 2800);
        } else {
            revealGem(btn);
            updateMultiplierDisplay(d.multiplier);
            if (gameMsg) gameMsg.textContent = `${d.safeCount} veilig · ${d.multiplier.toFixed(2)}× multiplier`;
            if (cashoutBtn) cashoutBtn.disabled = false;
            if (window.Sound) Sound.playCard();

            enableUnrevealedTiles();

            // All safe tiles found → auto cashout
            if (d.safeCount >= 25 - mineCount) {
                await cashOut();
            }
        }
    } catch {
        showError('Verbindingsfout');
        enableUnrevealedTiles();
    }
}

async function cashOut() {
    if (!gameActive || !gameId) return;

    gameActive = false;
    disableAllTiles();
    if (cashoutBtn) cashoutBtn.disabled = true;

    try {
        const d = await api('/api/mines/cashout.json', { gameId });
        if (d.error) { showError(d.error); return; }

        if (gameMsg) gameMsg.textContent = `✓ Uitbetaald: ${d.winAmount.toLocaleString('nl-NL')} EC (${d.multiplier}`;
        if (window.Sound) { Sound.playWin(); Sound.playCashRegister(); }

        d.minePositions?.forEach(pos => revealMine(getTile(pos), false));
    }
}

```


multiplayer.js

src/scripts/multiplayer.js

[illegible]


```

        if (myZone) myZone.classList.toggle('active-turn', isMyTurn);
    }

    renderControls(msg, myPlayer, myIndex);
}

function renderOtherPlayers(others, currentPlayerIndex, allPlayers, phase) {
    if (!otherPlayersLeft || !otherPlayersRight) return;
    otherPlayersLeft.innerHTML = '';
    otherPlayersRight.innerHTML = '';

    others.forEach((player, i) => {
        const globalIndex = allPlayers.findIndex(p => p.username === player.username);
        const isCurrentTurn = phase === 'playing' && currentPlayerIndex === globalIndex;

        const statusLabels = { waiting: 'Wachten', betting: 'Inzetten...', ready: 'Klaar', playing: 'Speelt', stood: 'Staat' };
        const resultLabels = { win: 'Gewonnen!', lose: 'Verloren', push: 'Gelijkspel' };
        const statusText = player.result
            ? resultLabels[player.result] || player.status
            : statusLabels[player.status] || player.status;

        const div = document.createElement('div');
        div.className = 'other-player' + (isCurrentTurn ? ' active-turn' : '');
        div.innerHTML = `
            <div class="other-player-name">${player.username}${player.isHost ? ' ■' : ''}</div>
            <div class="other-hand"></div>
            <div class="other-meta">
                <span class="other-score score-badge">${player.handValue} > 21 ? ' bust' : ''</span>${player.handValue}
                <span class="other-status">${statusText}${player.bet ? ` · ${player.bet} EC` : ''}</span>
            </div>
        `;

        const handEl = div.querySelector('.other-hand');
        player.hand.forEach((card, idx) => {
            const el = makeCardEl(card, false, true);
            el.style.animationDelay = `${idx * 80}ms`;
            handEl.appendChild(el);
        });

        (i % 2 === 0 ? otherPlayersLeft : otherPlayersRight).appendChild(div);
    });
}

function renderControls(msg, myPlayer, myIndex) {
    [ctrlWaiting, ctrlBetting, ctrlPlaying, ctrlResults].forEach(el => {
        if (el) el.style.display = 'none';
    });

    switch (msg.phase) {
        case 'waiting': showWaitingControls(msg); break;
        case 'betting': showBettingControls(myPlayer); break;
        case 'playing':
        case 'dealer': showPlayingControls(msg, myPlayer, myIndex); break;
        case 'results': showResultsControls(msg, myPlayer); break;
    }
}

function showWaitingControls(msg) {
    if (!ctrlWaiting) return;
    ctrlWaiting.style.display = 'flex';

    if (playerListEl) {

```

```

    playerListEl.innerHTML = msg.players
      .map(p => `<span class="player-chip">${p.username}${p.isHost ? ' ■' : ''}</span>`)
      .join('');
  }

  if (startBtn) startBtn.style.display = isHost ? 'inline-block' : 'none';
  if (waitMsg) {
    waitMsg.textContent = isHost ? 'Klaar om te starten!' : 'Wachten op de host...';
  }
}

function showBettingControls(myPlayer) {
  if (!ctrlBetting) return;
  ctrlBetting.style.display = 'flex';

  const alreadyBet = myPlayer?.status === 'ready';
  if (betBtn) betBtn.disabled = alreadyBet;
  if (betAmountEl) betAmountEl.disabled = alreadyBet;
  if (betWaitMsg) {
    betWaitMsg.textContent = alreadyBet
      ? 'Inzet geplaatst – wachten op anderen...'
      : 'Plaats je inzet!';
  }
}

function showPlayingControls(msg, myPlayer, myIndex) {
  if (!ctrlPlaying) return;
  ctrlPlaying.style.display = 'flex';

  const isMyTurn = msg.phase === 'playing' && msg.currentPlayerIndex === myIndex;
  if (hitBtn) hitBtn.disabled = !isMyTurn;
  if (standBtn) standBtn.disabled = !isMyTurn;

  if (turnMsg) {
    if (msg.phase === 'dealer') {
      turnMsg.textContent = 'Dealer speelt...';
    } else if (isMyTurn) {
      turnMsg.textContent = 'Jouw beurt!';
    } else {
      const cur = msg.players[msg.currentPlayerIndex];
      turnMsg.textContent = cur ? `${cur.username} is aan de beurt...` : 'Wachten...';
    }
  }
}

function showResultsControls(msg, myPlayer) {
  if (!ctrlResults) return;
  ctrlResults.style.display = 'flex';

  if (myPlayer && resultMsg) {
    const labels = { win: 'Gewonnen!', lose: 'Verloren', push: 'Gelijkspel' };
    let text = labels[myPlayer.result] || '-';
    if (myPlayer.winAmount > 0) text += ` (+${myPlayer.winAmount} EC)`;
    resultMsg.textContent = text;
    resultMsg.className = 'outcome-msg visible ' + (myPlayer.result || 'push');
  }

  if (playAgainBtn) playAgainBtn.style.display = isHost ? 'inline-block' : 'none';
  loadBalance();
}

function showError(message) {

```


src/scripts/poker.js

[illegible]


```

    if (myLabel) myLabel.textContent = myPlayer.username + (myPlayer.isHost ? ' ■' : '');

    const isMyTurn = ['preflop', 'flop', 'turn', 'river'].includes(msg.phase) && msg.currentPlayerIndex === myIndex;
    if (myZone) myZone.classList.toggle('active-turn', isMyTurn);
  }

  renderControls(msg, myPlayer, myIndex);
}

function renderOtherPlayers(others, currentPlayerIndex, allPlayers, phase) {
  if (!otherPlayersLeft || !otherPlayersRight) return;
  otherPlayersLeft.innerHTML = '';
  otherPlayersRight.innerHTML = '';

  const playingPhases = ['preflop', 'flop', 'turn', 'river'];
  const isResults = phase === 'results';

  others.forEach((player, i) => {
    const globalIndex = allPlayers.findIndex(p => p.username === player.username);
    const isCurrentTurn = playingPhases.includes(phase) && currentPlayerIndex === globalIndex;
    const isFolded = player.status === 'folded';

    const statusMap = {
      waiting: 'Wachten', antes: 'Inzetten...', ready: 'Klaar ✓',
      playing: 'Speelt', folded: 'Gefolded',
    };
    const resultMap = { win: 'Gewonnen! ■', lose: 'Verloren', tie: 'Gelijkspel' };
    const statusText = player.result
      ? (resultMap[player.result] || player.status)
      : (statusMap[player.status] || player.status);

    const betBadge = player.streetBet > 0
      ? `<span class="player-chip">${player.streetBet} EC</span>`
      : player.ante > 0
      ? `<span class="player-chip">Ante: ${player.ante}</span>`
      : '';

    const handNameBadge = isResults && player.handResult && !isFolded
      ? `<div class="hand-name-small">${player.handResult.name}</div>`
      : '';

    const div = document.createElement('div');
    div.className = 'other-player'
      + (isCurrentTurn ? ' active-turn' : '')
      + (isFolded ? ' folded' : '');

    div.innerHTML = `
      <div class="other-player-name">${player.username}${player.isHost ? ' ■' : ''}</div>
      <div class="other-hand"></div>
      ${handNameBadge}
      <div class="other-meta">
        <span class="other-status">${statusText}</span>
        ${betBadge}
      </div>
    `;

    const handEl = div.querySelector('.other-hand');

    if (isResults && player.hand?.length > 0 && !isFolded) {
      // Show revealed cards at results
      player.hand.forEach((card, idx) => {
        const el = makeCardEl(card, false, true);

```

```

        el.style.animationDelay = `${idx * 80}ms`;
        handEl.appendChild(el);
    });
} else if (playingPhases.includes(phase) && !isFolded && player.hasCards) {
    // Show 2 card backs during play
    for (let j = 0; j < 2; j++) {
        const el = makeCardEl(null, true, true);
        el.style.animationDelay = `${j * 80}ms`;
        handEl.appendChild(el);
    }
}

(i % 2 === 0 ? otherPlayersLeft : otherPlayersRight).appendChild(div);
});
}

function renderControls(msg, myPlayer, myIndex) {
    [ctrlWaiting, ctrlAntes, ctrlPlaying, ctrlResults].forEach(el => {
        if (el) el.style.display = 'none';
    });

    switch (msg.phase) {
        case 'waiting': showWaitingControls(msg); break;
        case 'antes': showAntesControls(myPlayer); break;
        case 'preflop':
        case 'flop':
        case 'turn':
        case 'river': showPlayingControls(msg, myPlayer, myIndex); break;
        case 'results': showResultsControls(msg, myPlayer); break;
    }
}

function showWaitingControls(msg) {
    if (!ctrlWaiting) return;
    ctrlWaiting.style.display = 'flex';

    if (playerListEl) {
        playerListEl.innerHTML = msg.players
            .map(p => `<span class="player-chip">${p.username}${p.isHost ? ' ■' : ''}</span>`)
            .join('');
    }

    const canStart = isHost && msg.players.length >= 2;
    if (startBtn) startBtn.style.display = canStart ? 'inline-block' : 'none';
    if (waitMsg) {
        waitMsg.textContent = isHost
            ? (msg.players.length < 2 ? 'Wachten op meer spelers (min. 2)...' : 'Klaar om te starten!')
            : 'Wachten op de host...';
    }
}

function showAntesControls(myPlayer) {
    if (!ctrlAntes) return;
    ctrlAntes.style.display = 'flex';

    const alreadyPlaced = myPlayer?.status === 'ready';
    if (anteBtn) anteBtn.disabled = alreadyPlaced;
    if (anteAmountEl) anteAmountEl.disabled = alreadyPlaced;
    if (anteWaitMsg) {
        anteWaitMsg.textContent = alreadyPlaced
            ? 'Inzet geplaatst – wachten op anderen...'
            : 'Plaats je inzet voor deze hand!';
    }
}

```

```

    }
}

function showPlayingControls(msg, myPlayer, myIndex) {
    if (!ctrlPlaying) return;
    ctrlPlaying.style.display = 'flex';

    const isMyTurn = msg.currentPlayerIndex === myIndex && myPlayer?.status === 'playing';
    const callAmount = isMyTurn ? Math.max(0, msg.currentBet - (myPlayer?.streetBet || 0)) : 0;
    const canCheck = isMyTurn && callAmount === 0;

    if (foldBtn) foldBtn.disabled = !isMyTurn;
    if (raiseBtn) raiseBtn.disabled = !isMyTurn;
    if (raiseAmountEl) raiseAmountEl.disabled = !isMyTurn;

    if (checkCallBtn) {
        checkCallBtn.disabled = !isMyTurn;
        if (canCheck) {
            checkCallBtn.textContent = 'Check';
            checkCallBtn.className = 'btn-action btn-check';
            checkCallBtn.dataset.action = 'check';
        } else {
            checkCallBtn.textContent = `Call ${callAmount} EC`;
            checkCallBtn.className = 'btn-action btn-call';
            checkCallBtn.dataset.action = 'call';
        }
    }
}

if (turnMsg) {
    if (isMyTurn) {
        turnMsg.textContent = callAmount > 0
            ? `Jouw beurt – ${callAmount} EC te callen`
            : 'Jouw beurt!';
    } else {
        const cur = msg.players[msg.currentPlayerIndex];
        turnMsg.textContent = cur ? `${cur.username} is aan de beurt...` : 'Wachten...';
    }
}

function showResultsControls(msg, myPlayer) {
    if (!ctrlResults) return;
    ctrlResults.style.display = 'flex';

    if (myPlayer && resultMsg) {
        const labels = { win: 'Gewonnen!', lose: 'Verloren', tie: 'Gelijkspel' };
        let text = labels[myPlayer.result] || '-';
        if (myPlayer.winAmount > 0) text += ` (+${myPlayer.winAmount} EC)`;
        if (myPlayer.handResult?.name && myPlayer.handResult.name !== '-') {
            text += ` – ${myPlayer.handResult.name}`;
        }
        resultMsg.textContent = text;
        const cls = myPlayer.result === 'tie' ? 'push' : (myPlayer.result || 'push');
        resultMsg.className = 'outcome-msg visible ' + cls;
    }

    if (playAgainBtn) playAgainBtn.style.display = isHost ? 'inline-block' : 'none';
    loadBalance();
}

function showError(message) {
    if (!errorMsg) return;

```


roulette.js

src/scripts/roulette.js

```
const WHEEL_ORDER = [0,32,15,19,21,2,25,17,34,6,27,13,36,11,30,8,23,10,5,24,16,33,1,20,14,31,9,22,18,29,7,28];  
const RED_NUMS = new Set([1,3,5,7,9,12,14,16,18,19,21,23,25,27,30,32,34,36]);  
  
let bank = 0;  
let bets = {};  
let spinning = false;  
let currentAngle = -Math.PI / 2; // 0 at top on load  
  
const canvas = document.getElementById('wheelCanvas');  
const ctx = canvas.getContext('2d');  
const bankEl = document.getElementById('bank');  
const messageEl = document.getElementById('message');  
const resultNumEl = document.getElementById('resultNum');  
const resultDisplay = document.getElementById('resultDisplay');  
const totalBetDisplay = document.getElementById('totalBetDisplay');  
const spinBtn = document.getElementById('spinBtn');  
const clearBtn = document.getElementById('clearBtn');  
const chipInput = document.getElementById('chipAmount');  
  
// ■■ Auth ████████████████████████████████████████████████████████████████████████████████  
function getToken() {  
    try { return sessionStorage.getItem('token'); } catch(e) { return null; }  
}  
  
async function loadBalance() {  
    try {  
        const token = getToken();  
        if (!token) return;  
        const res = await fetch('/api/balance.json', { headers: { 'Authorization': 'Bearer ' + token } });  
        if (!res.ok) return;  
        const j = await res.json();  
        if (j && typeof j.balance === 'number') {  
            bank = j.balance;  
            bankEl.textContent = bank.toLocaleString('nl-NL');  
        }  
    } catch(e) { console.warn('loadBalance failed', e); }  
}  
  
function saveBalance() {  
    const token = getToken();  
    if (!token) return;  
    fetch('/api/balance.json', {  
        method: 'POST',  
        headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + token },  
        body: JSON.stringify({ balance: bank })  
    }).catch(e => console.warn('saveBalance failed', e));  
}  
  
// ■■ Wheel drawing ████████████████████████████████████████████████████████████████████████████████  
function sectorColor(n) {  
    if (n === 0) return '#1a7a3c';  
    return RED_NUMS.has(n) ? '#b01020' : '#111111';  
}  
  
function drawWheel(angle) {
```

```

const cy = H / 2;
const R = Math.min(W, H) / 2 - 5;
const innerR = R * 0.3;
const n = WHEEL_ORDER.length;
const step = (Math.PI * 2) / n;

ctx.clearRect(0, 0, W, H);

// Gold outer rim
ctx.beginPath();
ctx.arc(cx, cy, R + 4, 0, Math.PI * 2);
ctx.fillStyle = '#FFE600';
ctx.fill();

// Sectors
for (let i = 0; i < n; i++) {
  const startA = angle + i * step - step / 2;
  const endA = startA + step;
  ctx.beginPath();
  ctx.moveTo(cx, cy);
  ctx.arc(cx, cy, R, startA, endA);
  ctx.closePath();
  ctx.fillStyle = sectorColor(WHEEL_ORDER[i]);
  ctx.fill();
  ctx.strokeStyle = 'rgba(255,230,0,0.2)';
  ctx.lineWidth = 0.5;
  ctx.stroke();
}

// Number labels
const fontSize = Math.max(7, Math.floor(R * 0.088));
for (let i = 0; i < n; i++) {
  const midA = angle + i * step;
  const labelR = R * 0.73;
  const tx = cx + labelR * Math.cos(midA);
  const ty = cy + labelR * Math.sin(midA);

  ctx.save();
  ctx.translate(tx, ty);
  ctx.rotate(midA + Math.PI / 2);
  ctx.fillStyle = '#FFE600';
  ctx.font = `bold ${fontSize}px Open Sans, sans-serif`;
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText(String(WHEEL_ORDER[i]), 0, 0);
  ctx.restore();
}

// Hub
ctx.beginPath();
ctx.arc(cx, cy, innerR, 0, Math.PI * 2);
ctx.fillStyle = '#0a0e1e';
ctx.fill();
ctx.strokeStyle = '#FFE600';
ctx.lineWidth = 2;
ctx.stroke();

// CMD text
ctx.fillStyle = '#FFE600';
ctx.font = `bold ${Math.max(10, Math.floor(R * 0.125))}px Open Sans, sans-serif`;
ctx.textAlign = 'center';
ctx.textBaseline = 'middle';

```



```
if (net > 0) { text = `+${net.toLocaleString('nl-NL')} gewonnen!`; type = 'win'; }
else if (net < 0) { text = `${net.toLocaleString('nl-NL')} verloren`; type = 'lose'; }
else { text = 'Quitte'; type = 'push'; }

messageEl.className = 'outcome-msg ' + type;
messageEl.textContent = text;
void messageEl.offsetWidth;
messageEl.classList.add('visible');

if (window.Sound) {
    if (net > 0) { window.Sound.playWin(); window.Sound.playCashRegister(); }
    else if (net < 0) { window.Sound.playLose(); }
}

function hideMessage() {
    if (messageEl) messageEl.classList.remove('visible');
}

// ■ Spin
function doSpin() {
    if (spinning) return;
    if (getTotalBet() < 1) {
        messageEl.className = 'outcome-msg push';
        messageEl.textContent = 'Plaats eerst een inzet';
        void messageEl.offsetWidth;
        messageEl.classList.add('visible');
        return;
    }

    spinning = true;
    spinBtn.disabled = true;
    clearBtn.disabled = true;
    if (resultDisplay) resultDisplay.style.visibility = 'hidden';
    hideMessage();

    if (window.Sound) {
        window.Sound.playShuffle();
        window.Sound.say.noMoreBets();
    }

    const result = Math.floor(Math.random() * 37);
    const targetAngle = getTargetAngle(WHEEL_ORDER.indexOf(result));

    animate(targetAngle, 5000, () => {
        if (window.Sound) window.Sound.playSwoosh();

        const net = resolveAllBets(result);
        bankEl.textContent = bank.toLocaleString('nl-NL');
        showResult(result, net);
        saveBalance();

        clearBets();
        spinning = false;
        spinBtn.disabled = false;
        clearBtn.disabled = false;

        if (window.Sound) setTimeout(() => window.Sound.say.placeBets(), 1400);
    });
}

// ■ Event listeners
```


■ rules.js

src/scripts/rules.js

```
const panel = document.getElementById('panel');
const buttons = document.querySelectorAll('.game-btn');

// Pick game from URL hash on load
const initialSlug = location.hash.slice(1);
if (initialSlug) loadGame(initialSlug);

buttons.forEach(btn => {
  btn.addEventListener('click', () => {
    const slug = btn.dataset.slug;
    history.replaceState(null, '', '#' + slug);
    loadGame(slug);
  });
});

function setActive(slug) {
  buttons.forEach(b => b.classList.toggle('active', b.dataset.slug === slug));
}

async function loadGame(slug) {
  setActive(slug);
  showLoading();

  try {
    const res = await fetch(`/api/rules.json?game=${encodeURIComponent(slug)}`);
    const data = await res.json();

    if (!res.ok || data.error) { showError(data.error ?? 'Fout bij het laden'); return; }
    showArticle(data);
  } catch {
    showError('Geen verbinding met Wikipedia');
  }
}

function showLoading() {
  panel.innerHTML = `
    <div class="rules-loading">
      <div class="spinner"></div>
      Laden via Wikipedia...
    </div>`;
}

function showError(msg) {
  panel.innerHTML = `<p class="rules-error">■■ ${msg}</p>`;
}

function showArticle(data) {
  const thumbHtml = data.thumbnail
    ? ``
    : '';

  const sectionsHtml = (data.sections ?? []).map(sec => {
    // Split text into paragraphs
    const paragraphs = sec.text
      .split('\n')
      .filter(p => p.trim())
      .map(p => `<p>${esc(p)}</p>`)
      .join('');
  });
}
```

```

    if (sec.title === null) {
        // Intro – no heading, render directly
        return `<div class="section-intro">${paragraphs}</div>`;
    }

    const tag = sec.level <= 2 ? 'h3' : 'h4';
    return `
        <div class="article-section">
            <${tag} class="section-title">${esc(sec.title)}</${tag}>
            <div class="section-body">${paragraphs}</div>
        </div>`;
    }).join('');

panel.innerHTML = `
    <article class="rules-article">
        <div class="article-header">
            ${thumbHtml}
            <div class="article-meta">
                <h2 class="article-title">${esc(data.title)}</h2>
                <p class="article-source">Bron: Wikipedia (Nederlands)</p>
            </div>
        </div>

        <div class="article-body">
            ${sectionsHtml}
        </div>

        ${data.sourceUrl ? `
            <a class="wiki-link" href="${esc(data.sourceUrl)}" target="_blank" rel="noopener">
                ■ Volledig artikel op Wikipedia
            </a>`; ''}
    </article>`;
}

function esc(str) {
    return String(str ?? '')
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;');
}

```

shop-legacy.js

src/scripts/shop-legacy.js

```
const PACKAGES = [
  { id: 'p1', coins: 60, price_cents: 260, price_eur: '€2,60' },
  { id: 'p5', coins: 120, price_cents: 510, price_eur: '€5,10' },
  { id: 'p10', coins: 180, price_cents: 760, price_eur: '€7,60' }
];

function renderPackages(){
  const root = document.getElementById('packages');
  for(const p of PACKAGES){
    const el = document.createElement('div'); el.className = 'pkg';
    el.innerHTML = `
      <h3>${p.coins.toLocaleString()} coins</h3>
      <div class="coins">${p.coins.toLocaleString()}</div>
      <div class="price">Price: ${p.price_eur}</div>
      <button data-id="${p.id}">Buy</button>
    `;
    root.appendChild(el);
  }
}

async function createPayment(packageId){
  try{
    // include auth token if available so the server can attach the purchase to the user
    let headers = { 'Content-Type': 'application/json' };
    if(window.getAuthToken) {
      try{ const t = await window.getAuthToken(); if(t) headers['Authorization'] = `Bearer ${t}`; }catch(e){}
    }
    const res = await fetch('/api/create-payment.json', { method: 'POST', headers, body: JSON.stringify({ packageId }) });
    const j = await res.json();
    if(res.ok && j.paymentUrl){
      // redirect to payment URL (stub or real provider)
      window.location.href = j.paymentUrl;
    }else{
      alert('Failed to create payment: ' + (j.error || JSON.stringify(j)));
    }
  }catch(err){
    console.error(err); alert('Network error creating payment');
  }
}

function attach(){
  document.getElementById('packages').addEventListener('click', (e)=>{
    const btn = e.target.closest('button[data-id]');
    if(!btn) return;
    const id = btn.dataset.id;
    btn.disabled = true; btn.textContent = 'Creating...';
    createPayment(id).finally(()=>{ btn.disabled = false; btn.textContent = 'Buy'; });
  });
}

document.addEventListener('DOMContentLoaded', ()=>{ renderPackages(); attach(); });
```

shop.js

src/scripts/shop.js

```
if (new URLSearchParams(location.search).get('payment') === 'success') {
  document.getElementById('success-banner').style.display = 'flex';

  // Webhook als fallback: controleer zelf of de betaling verwerkt is
  const savedPaymentId = sessionStorage.getItem('pendingMolliePaymentId');
  if (savedPaymentId) {
    sessionStorage.removeItem('pendingMolliePaymentId');

    const token = sessionStorage.getItem('token');
    if (token) {
      fetch('/api/check-payment', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ' + token },
        body: JSON.stringify({ molliePaymentId: savedPaymentId })
      })
        .then(function(res) { return res.json(); })
        .then(function(data) {
          if (data.credited) {
            console.log('check-payment: ' + data.coins + ' coins bijgeschreven via fallback');
          }
        })
        .catch(function(err) {
          console.error('check-payment fout:', err);
        });
    }
  }
}

document.querySelectorAll('[data-pkg]').forEach(btn => {
  btn.addEventListener('click', async () => {
    const token = sessionStorage.getItem('token');
    const msg = document.getElementById('msg');
    if (!token) { location.href = '/login'; return; }
    btn.disabled = true; btn.textContent = 'Bezig...';
    msg.textContent = ''; msg.className = '';
    try {
      const res = await fetch('/api/create-payment', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ' + token },
        body: JSON.stringify({ packageId: btn.dataset.pkg })
      });
      const data = await res.json();
      if (res.ok && data.paymentUrl) {
        if (data.molliePaymentId) {
          sessionStorage.setItem('pendingMolliePaymentId', data.molliePaymentId);
        }
        location.href = data.paymentUrl;
      } else {
        msg.className = 'err'; msg.textContent = data.error || 'Betaling mislukt';
        btn.disabled = false; btn.textContent = 'Kopen';
      }
    } catch {
      msg.className = 'err'; msg.textContent = 'Netwerkfout';
      btn.disabled = false; btn.textContent = 'Kopen';
    }
  });
});
```

■ signup.js

src/scripts/signup.js

```
const btn = document.getElementById('btn');
const msg = document.getElementById('msg');

async function signup() {
  if (!btn || !msg) return;
  btn.disabled = true; btn.textContent = 'Bezig...';
  msg.textContent = ''; msg.className = '';
  const username = document.getElementById('username').value.trim();
  const password = document.getElementById('password').value;
  try {
    const res = await fetch('/api/signup', {
      method: 'POST', credentials: 'include',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });
    const data = await res.json();
    if (res.ok) {
      msg.className = 'ok'; msg.textContent = 'Account aangemaakt! Je wordt doorgestuurd...';
      setTimeout(() => { location.href = '/login'; }, 800);
    } else {
      msg.className = 'err'; msg.textContent = data.message || 'Registratie mislukt';
    }
  } catch {
    msg.className = 'err'; msg.textContent = 'Netwerkfout, probeer opnieuw';
  } finally {
    btn.disabled = false; btn.textContent = 'Account aanmaken';
  }
}

btn?.addEventListener('click', signup);
document.addEventListener('keydown', e => { if (e.key === 'Enter') signup(); });
```

■ sound.js

src/scripts/sound.js

```

/* Casino sound manager - Web Audio API + ElevenLabs TTS */
(function () {
    'use strict';

    const MUTE_KEY = 'bj_muted';
    let muted = localStorage.getItem(MUTE_KEY) === 'true';
    let audioCtx = null;

    // Raw ArrayBuffers pre-fetched before AudioContext is created
    const rawCache = new Map();
    // Decoded AudioBuffers ready to play
    const audioCache = new Map();

    function getCtx() {
        if (!audioCtx) {
            audioCtx = new (window.AudioContext || window.webkitAudioContext)();
        }
        if (audioCtx.state === 'suspended') audioCtx.resume();
        return audioCtx;
    }

    // ■■ Synthetic effects ████████████████████████████████████████████████████████████████████████████

    function playChip() {
        if (muted) return;
        try {
            const ctx = getCtx();
            const t = ctx.currentTime;
            for (let i = 0; i < 2; i++) {
                const osc = ctx.createOscillator();
                const gain = ctx.createGain();
                osc.connect(gain);
                gain.connect(ctx.destination);
                osc.type = 'triangle';
                const s = t + i * 0.045;
                osc.frequency.setValueAtTime(1600 - i * 200, s);
                osc.frequency.exponentialRampToValueAtTime(800, s + 0.07);
                gain.gain.setValueAtTime(0.22, s);
                gain.gain.exponentialRampToValueAtTime(0.001, s + 0.12);
                osc.start(s);
                osc.stop(s + 0.12);
            }
        } catch (e) {}
    }

    function playCard() {
        if (muted) return;
        try {
            const ctx = getCtx();
            const t = ctx.currentTime;
            const bufSize = Math.floor(ctx.sampleRate * 0.07);
            const buf = ctx.createBuffer(1, bufSize, ctx.sampleRate);
            const data = buf.getChannelData(0);
            for (let i = 0; i < bufSize; i++) {
                data[i] = (Math.random() * 2 - 1) * Math.pow(1 - i / bufSize, 2);
            }
            const src = ctx.createBufferSource();
            src.buffer = buf;

```

```

    const hpf    = ctx.createBiquadFilter();
    hpf.type     = 'highpass';
    hpf.frequency.value = 2200;
    const gain   = ctx.createGain();
    gain.gain.setValueAtTime(0.16, t);
    gain.gain.exponentialRampToValueAtTime(0.001, t + 0.08);
    src.connect(hpf);
    hpf.connect(gain);
    gain.connect(ctx.destination);
    src.start(t);
  } catch (e) {}
}

function playWin() {
  if (muted) return;
  try {
    const ctx  = getCtx();
    const t    = ctx.currentTime;
    const freqs = [523, 659, 784, 1047];
    freqs.forEach((freq, i) => {
      const osc = ctx.createOscillator();
      const gain = ctx.createGain();
      osc.connect(gain);
      gain.connect(ctx.destination);
      osc.type = 'sine';
      const s = t + i * 0.09;
      osc.frequency.value = freq;
      gain.gain.setValueAtTime(0, s);
      gain.gain.linearRampToValueAtTime(0.14, s + 0.02);
      gain.gain.exponentialRampToValueAtTime(0.001, s + 0.25);
      osc.start(s);
      osc.stop(s + 0.25);
    });
  } catch (e) {}
}

function playLose() {
  if (muted) return;
  try {
    const ctx  = getCtx();
    const t    = ctx.currentTime;
    const freqs = [392, 349, 311];
    freqs.forEach((freq, i) => {
      const osc = ctx.createOscillator();
      const gain = ctx.createGain();
      osc.connect(gain);
      gain.connect(ctx.destination);
      osc.type = 'sine';
      const s = t + i * 0.09;
      osc.frequency.value = freq;
      gain.gain.setValueAtTime(0, s);
      gain.gain.linearRampToValueAtTime(0.12, s + 0.02);
      gain.gain.exponentialRampToValueAtTime(0.001, s + 0.22);
      osc.start(s);
      osc.stop(s + 0.22);
    });
  } catch (e) {}
}

// Swoosh – hole card reveal
function playSwoosh() {
  if (muted) return;

```

```

try {
  const ctx = getCtx();
  const t = ctx.currentTime;
  const dur = 0.18;
  const bufSize = Math.floor(ctx.sampleRate * dur);
  const buf = ctx.createBuffer(1, bufSize, ctx.sampleRate);
  const data = buf.getChannelData(0);
  for (let i = 0; i < bufSize; i++) {
    data[i] = (Math.random() * 2 - 1) * Math.sin(Math.PI * i / bufSize);
  }
  const src = ctx.createBufferSource();
  src.buffer = buf;
  const bpf = ctx.createBiquadFilter();
  bpf.type = 'bandpass';
  bpf.frequency.setValueAtTime(3000, t);
  bpf.frequency.exponentialRampToValueAtTime(800, t + dur);
  bpf.Q.value = 1.5;
  const gain = ctx.createGain();
  gain.gain.setValueAtTime(0.28, t);
  gain.gain.exponentialRampToValueAtTime(0.001, t + dur);
  src.connect(bpf);
  bpf.connect(gain);
  gain.connect(ctx.destination);
  src.start(t);
} catch (e) {}
}

// Card rattle – shuffle
function playShuffle() {
  if (muted) return;
  try {
    const ctx = getCtx();
    const t = ctx.currentTime;
    for (let i = 0; i < 8; i++) {
      const bufSize = Math.floor(ctx.sampleRate * 0.04);
      const buf = ctx.createBuffer(1, bufSize, ctx.sampleRate);
      const data = buf.getChannelData(0);
      for (let j = 0; j < bufSize; j++) {
        data[j] = (Math.random() * 2 - 1) * Math.pow(1 - j / bufSize, 1.5);
      }
      const src = ctx.createBufferSource();
      src.buffer = buf;
      const hpf = ctx.createBiquadFilter();
      hpf.type = 'highpass';
      hpf.frequency.value = 1800;
      const gain = ctx.createGain();
      const s = t + i * 0.065 + Math.random() * 0.02;
      gain.gain.setValueAtTime(0.13 + Math.random() * 0.06, s);
      gain.gain.exponentialRampToValueAtTime(0.001, s + 0.05);
      src.connect(hpf);
      hpf.connect(gain);
      gain.connect(ctx.destination);
      src.start(s);
    }
  } catch (e) {}
}

// Cash register ding – win payout
function playCashRegister() {
  if (muted) return;
  try {
    const ctx = getCtx();

```


■ vakken.js

src/scripts/vakken.js

```
const TYPE_LABELS = {
  vak: 'Vak', project: 'Project', stage: 'Stage', minor: 'Minor'
};

let allCourses = [];
let purchased = [];
let balance = 0;
let totalEc = 0;

async function load() {
  const token = sessionStorage.getItem('token');
  if (!token) { location.href = '/login'; return; }

  const [coursesRes, meRes] = await Promise.all([
    fetch('/api/courses.json', { headers: { Authorization: 'Bearer ' + token } }),
    fetch('/api/me', { headers: { Authorization: 'Bearer ' + token } })
  ]);

  if (!coursesRes.ok) { location.href = '/login'; return; }

  const data = await coursesRes.json();
  const me = meRes.ok ? await meRes.json() : {};

  allCourses = data.courses;
  purchased = data.purchased ?? [];
  balance = me.coins_amount ?? 0;
  totalEc = data.totalEc;

  render();
}

function render() {
  document.getElementById('hdr-balance').textContent = balance.toLocaleString('nl-NL');

  const earnedEc = allCourses
    .filter(c => purchased.includes(c.id))
    .reduce((s, c) => s + c.ec, 0);
  const pct = totalEc > 0 ? Math.round((earnedEc / totalEc) * 100) : 0;

  document.getElementById('progress-label').textContent = `${earnedEc} / ${totalEc} EC behaald`;
  document.getElementById('progress-pct').textContent = `${pct}%`;
  document.getElementById('progress-fill').style.width = pct + '%';

  const diploma = document.getElementById('diploma-banner');
  diploma.style.display =
    purchased.length === allCourses.length && allCourses.length > 0 ? 'flex' : 'none';

  const jaren = {};
  for (const c of allCourses) {
    if (!jaren[c.jaar]) jaren[c.jaar] = [];
    jaren[c.jaar].push(c);
  }

  const container = document.getElementById('jaren-container');
  container.innerHTML = '';

  for (const jaar of Object.keys(jaren).map(Number).sort()) {
    const section = document.createElement('div');
```

```

    section.className = 'jaar-section';

    const label = document.createElement('div');
    label.className = 'jaar-label';
    label.textContent = `Jaar ${jaar}`;
    section.appendChild(label);

    const grid = document.createElement('div');
    grid.className = 'courses-grid';
    for (const course of jaren[jaar]) grid.appendChild(buildCard(course));

    section.appendChild(grid);
    container.appendChild(section);
  }
}

function buildCard(course) {
  const owned = purchased.includes(course.id);
  const card = document.createElement('div');
  card.className = 'course-card' + (owned ? ' owned' : '');

  const top = document.createElement('div');
  top.className = 'card-top';

  const name = document.createElement('div');
  name.className = 'course-name';
  name.textContent = course.name;

  const badge = document.createElement('span');
  badge.className = `type-badge type-${course.type}`;
  badge.textContent = TYPE_LABELS[course.type] ?? course.type;

  top.appendChild(name);
  top.appendChild(badge);

  const bottom = document.createElement('div');
  bottom.className = 'card-bottom';

  const ec = document.createElement('div');
  ec.className = 'ec-label';
  ec.innerHTML = `<b>${course.ec}</b> EC`;

  if (owned) {
    const tag = document.createElement('span');
    tag.className = 'owned-tag';
    tag.textContent = '✓ Gehaald';
    bottom.appendChild(ec);
    bottom.appendChild(tag);
  } else {
    const btn = document.createElement('button');
    btn.className = 'buy-btn';
    btn.textContent = 'Kopen';
    btn.disabled = balance < course.ec;
    btn.addEventListener('click', () => buyClick(course, btn));
    bottom.appendChild(ec);
    bottom.appendChild(btn);
  }

  card.appendChild(top);
  card.appendChild(bottom);
  return card;
}

```

```

async function buyClick(course, btn) {
  const token = sessionStorage.getItem('token');
  const msg = document.getElementById('global-msg');
  btn.disabled = true; btn.textContent = 'Bezig...';
  msg.textContent = ''; msg.className = '';

  try {
    const res = await fetch('/api/buy-course', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ' + token },
      body: JSON.stringify({ courseId: course.id }),
    });
    const data = await res.json();
    if (res.ok) {
      purchased.push(course.id);
      balance -= course.ec;
      render();
    } else {
      msg.className = 'err';
      msg.textContent = data.error || 'Kopen mislukt';
      btn.disabled = false; btn.textContent = 'Kopen';
    }
  } catch {
    msg.className = 'err'; msg.textContent = 'Netwerkfout';
    btn.disabled = false; btn.textContent = 'Kopen';
  }
}

load();

```